

LINKED LIST

A complete placement + competitive programming guide to linked lists — covering all fundamental operations, advanced problems, and interview patterns.

SINGLY LINKED LIST

1. Overview

A **singly linked list** is a linear data structure where each element (node) stores a value and a pointer/reference to the next node in the sequence. Unlike arrays, elements are **not stored contiguously** in memory. The list ends when a node's **next** pointer is **nullptr** (or **None** in Python).

```
[data | next] → [data | next] → [data | next] → NULL
```

2. Intuition

Think of a singly linked list like a **treasure hunt** — each clue points to the next clue, and you can only move forward. You cannot go backward because there is no previous pointer.

- **Why it works:** Each node is independent. You can insert/delete nodes without shifting everything else, unlike arrays.
- **Trade-off:** Random access (`arr[i]`) is impossible — you must traverse from the head to reach any node.

3. When to Use It

- You need **frequent insertions/deletions** at arbitrary positions (especially at the head).
- You don't know the size in advance and need dynamic growth.
- You want to avoid memory reallocation (like **vector** resizing).
- You're implementing stacks, queues, or adjacency lists for graphs.

Trigger phrases:

- "Insert/delete at front/back"
- "Dynamic collection"
- "No random access needed"
- "Memory efficient insertions"

4. When Not to Use It

- When you need **random access** by index → use array/vector.
- When you frequently access the **last element** → doubly linked list or array.
- When memory overhead per element is a concern (each node stores an extra pointer).

- When cache locality matters (arrays are cache-friendly; linked lists are not).
- When you need **binary search** or **sorting** (arrays are better).

5. Core Concepts

Concept	Explanation	Why It Matters
Node	A struct/class with <code>data</code> and <code>next</code> pointer	Building block of the list
Head	Pointer to the first node	Entry point to the list
Tail	Last node whose <code>next = NULL</code>	End-of-list marker
Traversal	Walk from head to tail via <code>next</code>	Only way to access elements
Insertion	Create a new node, adjust pointers	O(1) at head, O(n) at tail
Deletion	Bypass the target node	O(1) at head, O(n) otherwise

6. Step-by-Step Algorithm

Insert at Head

1. Create a new node.
2. Set `newNode->next = head`.
3. Update `head = newNode`.

Insert at Tail

1. Create a new node with `next = NULL`.
2. If list is empty, set `head = newNode`.
3. Else traverse to last node, set `last->next = newNode`.

Delete at Head

1. If list is empty, return.
2. Save `temp = head`.
3. Set `head = head->next`.
4. Delete `temp`.

Delete a Value

1. If head holds the value, delete head.
2. Traverse with `prev` and `curr` pointers.
3. When `curr->data == value`, set `prev->next = curr->next`.
4. Delete `curr`.

Search

1. Start from head.
2. Move `curr = curr->next` until value found or end.

7. Dry Run

Insert 10, 20, 30 at head:

Step	Operation	List State
0	Initial	NULL
1	Insert 10 at head	10 → NULL
2	Insert 20 at head	20 → 10 → NULL
3	Insert 30 at head	30 → 20 → 10 → NULL

Delete 20:

Step	Operation	List State
0	Initial	30 → 20 → 10 → NULL
1	Traverse, prev=30, curr=20	30 → 20 → 10 → NULL
2	Set <code>prev->next = curr->next</code>	30 → 10 → NULL
3	Delete curr(20)	30 → 10 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyLinkedList {
private:
    Node* head;

public:
    SinglyLinkedList() : head(nullptr) {}

    void insertAtHead(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }

    void insertAtTail(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
        }
    }
};
```

```
        return;
    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

void deleteValue(int val) {
    if (!head) return;
    if (head->data == val) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }
    Node* curr = head;
    while (curr->next && curr->next->data != val)
        curr = curr->next;
    if (curr->next) {
        Node* temp = curr->next;
        curr->next = curr->next->next;
        delete temp;
    }
}

bool search(int val) {
    Node* temp = head;
    while (temp) {
        if (temp->data == val) return true;
        temp = temp->next;
    }
    return false;
}

void print() {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

~SinglyLinkedList() {
    Node* temp;
    while (head) {
        temp = head;
        head = head->next;
        delete temp;
    }
}

};

int main() {
```

```

SinglyLinkedList list;
list.insertAtHead(10);
list.insertAtHead(20);
list.insertAtTail(30);
list.print(); // 20 -> 10 -> 30 -> NULL
list.deleteValue(10);
list.print(); // 20 -> 30 -> NULL
cout << list.search(30) << endl; // 1
return 0;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_head(self, val):
        new_node = Node(val)
        new_node.next = self.head
        self.head = new_node

    def insert_at_tail(self, val):
        new_node = Node(val)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def delete_value(self, val):
        if not self.head:
            return
        if self.head.data == val:
            self.head = self.head.next
            return
        curr = self.head
        while curr.next and curr.next.data != val:
            curr = curr.next
        if curr.next:
            curr.next = curr.next.next

    def search(self, val):
        temp = self.head

```

```

        while temp:
            if temp.data == val:
                return True
            temp = temp.next
        return False

    def print_list(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("NULL")

# Example
ll = SinglyLinkedList()
ll.insert_at_head(10)
ll.insert_at_head(20)
ll.insert_at_tail(30)
ll.print_list() # 20 -> 10 -> 30 -> NULL
ll.delete_value(10)
ll.print_list() # 20 -> 30 -> NULL
print(ll.search(30)) # True

```

10. Code Explanation

- **Node struct/class:** Stores `data` and `next` pointer. Constructor initializes data and sets `next` to null.
- **insertAtHead:** $O(1)$. New node points to current head, then head is updated.
- **insertAtTail:** $O(n)$ because we traverse to the end. Maintain a tail pointer for $O(1)$ optimization.
- **deleteValue:** Two cases — head deletion ($O(1)$) or middle/end deletion ($O(n)$). Uses a `curr` pointer to find the node just before the target.
- **search:** Linear scan until value found or end reached.
- **Destructor:** Iterates through all nodes, deleting each to prevent memory leaks.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$	$O(1)$
Insert at position	$O(n)$	$O(1)$
Delete at head	$O(1)$	$O(1)$
Delete by value	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$
Traversal	$O(n)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach	Example
Two-pointer	Need to find middle, detect cycle, or find nth from end	Use slow/fast pointers	Middle of LL
Dummy node	Head may change during operation	Create a dummy node pointing to head	Merge lists, remove nth
Reversal	Need to reverse sublist or whole list	Iterative 3-pointer or recursive	Reverse list
Runner technique	Need to process with offset	Move one pointer ahead by k steps	Remove nth from end

13. Common Mistakes

- **Losing the head:** Always update head correctly when inserting/deleting at front.
- **Dangling pointers:** When deleting, don't forget to `delete` the node in C++.
- **Null pointer dereference:** Always check `if (!head)` before accessing `head->next`.
- **Infinite loops:** Forgetting to advance `temp = temp->next` in a loop.
- **Memory leaks:** In C++, always delete nodes. In modern C++, consider `unique_ptr`.

14. Edge Cases

- Empty list (`head == nullptr`): All operations must handle this gracefully.
- Single element: Inserting/deleting may change head to null.
- Two elements: Many bugs surface here (e.g., reversal with 2 nodes).
- Duplicate values: `deleteValue` typically deletes first occurrence only.
- Value not found: Search should return false; delete should do nothing.

15. Variations

Variation	Change	Use Case
Doubly linked list	Each node has <code>prev</code> and <code>next</code>	Easier backward traversal, browser history
Circular linked list	Tail's <code>next</code> points to head	Round-robin scheduling, Josephus problem
XOR linked list	Uses XOR of prev/next addresses	Memory-efficient (not for interviews)

16. Related Data Structures

Structure	Comparison
Array/Vector	$O(1)$ random access, $O(n)$ insert/delete; linked list is opposite
Doubly Linked List	All singly operations + $O(1)$ tail deletion, backward traversal
Stack/Queue	Often implemented using linked lists for dynamic size

Structure	Comparison
Deque	Dynamic array of pointers; better cache performance than linked list

17. Practice Problems

Easy

- **Reverse Linked List** — LeetCode 206 — Iterative reversal
- **Middle of the Linked List** — LeetCode 876 — Slow/fast pointer

Medium

- **Add Two Numbers** — LeetCode 2 — Traverse both lists, maintain carry
- **Remove Nth Node From End** — LeetCode 19 — Two-pointer with offset
- **Sort List** — LeetCode 148 — Merge sort on linked list

Hard

- **Reverse Nodes in k-Group** — LeetCode 25 — Reverse segments iteratively
- **Merge k Sorted Lists** — LeetCode 23 — Min-heap or divide & conquer

18. Interview Explanation

"A singly linked list is a linear collection of nodes where each node stores data and a pointer to the next node. It supports $O(1)$ insertion at the head and $O(n)$ traversal, but no random access. I use it when I need dynamic memory allocation without reallocation overhead, or when implementing stacks, queues, or adjacency lists. Common interview patterns include two-pointer techniques, dummy nodes, and reversal."

19. Revision Notes

- Singly linked list: `Node { data, next }`, head pointer.
- No random access. Traversal is the only way to reach elements.
- Dummy node (`Node* dummy = new Node(0); dummy->next = head;`) simplifies edge cases.
- Always check for null before dereferencing.
- Insert at head: $O(1)$. Insert at tail: $O(n)$ unless tail pointer maintained.
- Deletion: use `prev->next = curr->next`.
- Two-pointer technique solves many problems in $O(n)$.

20. Final Cheat Sheet

SINGLY LINKED LIST

When to use: Dynamic insertions/deletions, no random access needed

Main ops: `insertAtHead` $O(1)$, `insertAtTail` $O(n)$, `delete` $O(n)$, `search` $O(n)$

Key code: `Node* temp = head; while(temp) { ... temp = temp->next; }`

Edge cases: Empty list, single node, head modification
Traps: Null pointer, losing head, memory leaks, infinite loops

DOUBLY LINKED LIST

1. Overview

A **doubly linked list (DLL)** is a linear data structure where each node contains three fields: a value, a pointer to the **next** node, and a pointer to the **previous** node. This allows traversal in both directions.

NULL $\leftarrow$ [prev | data | next] $\leftrightarrow$ [prev | data | next] $\leftrightarrow$ [prev | data | next] $\rightarrow$ NULL

2. Intuition

Think of a doubly linked list like a **playlist with next and previous buttons** — you can move forward and backward freely. The extra **prev** pointer gives you bidirectional traversal at the cost of one extra pointer per node.

- **Why it works:** The **prev** pointer makes operations at the tail (delete, insert) $O(1)$ instead of $O(n)$.
- **Trade-off:** Each node uses more memory (extra pointer), and insert/delete operations need to update two pointers instead of one.

3. When to Use It

- You need to traverse **backward** efficiently.
- You frequently **delete the last node** ($O(1)$ with tail pointer).
- You need to implement **LRU cache** or **browser history**.
- You need to **insert before a given node** without traversing from head.
- You need $O(1)$ operations at both ends (deque).

Trigger phrases:

- "Backward traversal"
- "Delete from end"
- "Insert before a node"
- "LRU cache"
- "Browser history"

4. When Not to Use It

- Memory is constrained (DLL uses 2x pointers per node vs singly linked list).
- You only need forward traversal (singly linked list is sufficient).
- You need random access (use array/vector).
- Cache locality is critical (arrays are better).

5. Core Concepts

Concept	Explanation	Why It Matters
Node	Stores <code>data</code> , <code>next</code> , <code>prev</code>	Building block
Head	First node, <code>prev = NULL</code>	Entry point
Tail	Last node, <code>next = NULL</code>	O(1) backward operations
Bidirectional traversal	Move forward via <code>next</code> , backward via <code>prev</code>	Reverse iteration, palindrome check
Two-pointer updates	Both <code>next</code> and <code>prev</code> must be updated	More complex insert/delete logic

6. Step-by-Step Algorithm

Insert at Head

1. Create new node.
2. `newNode->next = head`.
3. `newNode->prev = NULL`.
4. If head exists, `head->prev = newNode`.
5. `head = newNode`.

Insert at Tail

1. Create new node. `newNode->next = NULL`.
2. If list is empty, set `head = newNode`, `newNode->prev = NULL`.
3. Else traverse to tail, `tail->next = newNode`.
4. `newNode->prev = tail`.

Delete a Node (given pointer)

1. If node is head, `head = node->next`.
2. If `node->next` exists, `node->next->prev = node->prev`.
3. If `node->prev` exists, `node->prev->next = node->next`.
4. Delete node.

7. Dry Run

Insert 10, 20, 30 at head:

Step	Operation	List State
0	Initial	<code>NULL</code>
1	Insert 10	<code>NULL ← 10 → NULL</code>
2	Insert 20	<code>NULL ← 20 ↔ 10 → NULL</code>
3	Insert 30	<code>NULL ← 30 ↔ 20 ↔ 10 → NULL</code>

Delete node 20 (given pointer):

Step	Operation	List State
0	Initial	NULL ← 30 ↔ 20 ↔ 10 → NULL
1	20->prev->next = 20->next	30->next = 10
2	20->next->prev = 20->prev	10->prev = 30
3	Delete 20	NULL ← 30 ↔ 10 → NULL

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
    Node(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class DoublyLinkedList {
private:
    Node* head;
    Node* tail;

public:
    DoublyLinkedList() : head(nullptr), tail(nullptr) {}

    void insertAtHead(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = tail = newNode;
            return;
        }
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    void insertAtTail(int val) {
        Node* newNode = new Node(val);
        if (!tail) {
            head = tail = newNode;
            return;
        }
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
}

```

```

void deleteNode(Node* del) {
    if (!head || !del) return;
    if (del == head) head = del->next;
    if (del == tail) tail = del->prev;
    if (del->prev) del->prev->next = del->next;
    if (del->next) del->next->prev = del->prev;
    delete del;
}

void deleteValue(int val) {
    Node* temp = head;
    while (temp) {
        if (temp->data == val) {
            deleteNode(temp);
            return;
        }
        temp = temp->next;
    }
}

void printForward() {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

void printBackward() {
    Node* temp = tail;
    while (temp) {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}

~DoublyLinkedList() {
    Node* temp;
    while (head) {
        temp = head;
        head = head->next;
        delete temp;
    }
}

};

```

9. Python Implementation

```
class Node:
    def __init__(self, val):
        self.data = val
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_at_head(self, val):
        new_node = Node(val)
        if not self.head:
            self.head = self.tail = new_node
            return
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

    def insert_at_tail(self, val):
        new_node = Node(val)
        if not self.tail:
            self.head = self.tail = new_node
            return
        self.tail.next = new_node
        new_node.prev = self.tail
        self.tail = new_node

    def delete_node(self, node):
        if not self.head or not node:
            return
        if node == self.head:
            self.head = node.next
        if node == self.tail:
            self.tail = node.prev
        if node.prev:
            node.prev.next = node.next
        if node.next:
            node.next.prev = node.prev

    def delete_value(self, val):
        temp = self.head
        while temp:
            if temp.data == val:
                self.delete_node(temp)
                return
            temp = temp.next

    def print_forward(self):
        temp = self.head
        while temp:
```

```
        print(temp.data, end=" <-> ")
        temp = temp.next
    print("NULL")

    def print_backward(self):
        temp = self.tail
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.prev
        print("NULL")
```

10. Code Explanation

- **Two pointers:** `head` and `tail` are maintained for $O(1)$ operations at both ends.
- **insertAtHead/insertAtTail:** Update both `next` and `prev` pointers of the new node and its neighbor.
- **deleteNode:** Given a pointer, adjusts both `prev->next` and `next->prev`. Handles head/tail cases.
- **printBackward:** Uses `tail` and `prev` pointers to traverse in reverse $O(n)$.
- **Edge cases:** Empty list, single node, head/tail deletion are all handled explicitly.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(1)$	$O(1)$
Delete given node	$O(1)$	$O(1)$
Delete by value	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$
Forward traversal	$O(n)$	$O(1)$
Backward traversal	$O(n)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
LRU Cache	"Least recently used", "cache eviction"	DLL + hashmap, move accessed node to front
Browser history	"Back/forward navigation"	DLL with current pointer
Deque implementation	$O(1)$ push/pop both ends	DLL with head and tail
Music playlist	"Next/previous track"	Circular DLL or regular DLL

13. Common Mistakes

- **Forgetting to update both `prev` and `next`:** Always update both pointers.
- **Not updating `tail`:** When deleting the last node, update tail.
- **Not updating `head`:** When deleting the first node, update head.
- **Null pointer in `prev->next` or `next->prev`:** Check for null before dereferencing.

14. Edge Cases

- Empty list: All operations should handle null head and tail.
- Single node: Deleting it should make both head and tail null.
- Two nodes: Deleting head or tail must update the other's pointers.
- Value not found: Delete should traverse to end and do nothing.

15. Variations

Variation	Change	Use Case
Circular DLL	Tail's <code>next</code> points to head, head's <code>prev</code> points to tail	Music playlist, round-robin
DLL with sentinel	Dummy nodes at both ends	Simplifies boundary conditions

16. Related Data Structures

Structure	Comparison
Singly Linked List	Less memory, but no backward traversal, $O(n)$ tail operations
Deque (STL)	Dynamic array of pointers; better cache performance, similar API
Array	$O(1)$ random access, but $O(n)$ insert/delete at front

17. Practice Problems

Easy

- **Design Browser History** — LeetCode 1472 — DLL with current pointer
- **Flatten a Multilevel Doubly Linked List** — LeetCode 430 — Recursive flattening

Medium

- **LRU Cache** — LeetCode 146 — DLL + hashmap
- **Design Linked List** — LeetCode 707 — Full DLL implementation

Hard

- **LFU Cache** — LeetCode 460 — Frequency-based eviction with DLLs

18. Interview Explanation

"A doubly linked list stores prev and next pointers in each node, allowing O(1) operations at both ends and backward traversal. It's my go-to for implementing LRU caches, deques, and browser history. The key trade-off is extra memory per node for the prev pointer. In interviews, I use a dummy head/tail to simplify boundary conditions."

19. Revision Notes

- DLL: `Node { data, prev, next }`, maintain `head` and `tail`.
- Insert at either end: O(1). Delete given node: O(1).
- Always update both `prev` and `next` pointers.
- Dummy nodes simplify boundary conditions.
- Used in LRU cache, browser history, deque.

20. Final Cheat Sheet

DOUBLY LINKED LIST

When to use: Bidirectional traversal, O(1) tail ops, cache implementation

Main ops: `insertAtHead` O(1), `insertAtTail` O(1), `deleteNode` O(1)

Key code: `node->next->prev = node->prev; node->prev->next = node->next;`

Edge cases: Empty list, single node, head/tail deletion

Traps: Forgetting to update both pointers, null pointer dereference

REVERSE LINKED LIST

1. Overview

Reverse a linked list means reversing the direction of the `next` pointers so that the tail becomes the head and vice versa. This is one of the most fundamental linked list operations.

Before: `1 → 2 → 3 → 4 → NULL`

After: `NULL ← 1 ← 2 ← 3 ← 4`

Result: `4 → 3 → 2 → 1 → NULL`

2. Intuition

Think of it like **reversing a stack of papers** — you pick up one sheet at a time and place it on top of a new pile. The iterative approach uses three pointers (`prev`, `curr`, `next`) to reverse links one at a time.

- **Why it works:** Each node's `next` is reassigned to point to the previous node. By walking through the list once, we reverse all pointers.
- **Recursive version:** The recursive approach reverses the rest of the list first, then makes the current node's `next->next` point back to the current node.

3. When to Use It

- A problem explicitly asks "reverse a linked list".
- You need to check if a list is a palindrome (reverse and compare halves).
- You need to reverse a sublist or k-group.
- You need to process a list from tail to head (without extra space).
- You're implementing a stack using a linked list (push/pop at head).

Trigger phrases:

- "Reverse the linked list"
- "Palindrome linked list"
- "Reverse nodes in k-group"
- "Reverse between positions"

4. When Not to Use It

- If you only need to **read** the list in reverse order, consider using a stack ($O(n)$ space, simpler code).
- If the list is very large and recursion depth is a concern, use iterative.
- If you need to preserve the original list, make a copy first.

5. Core Concepts

Concept	Explanation	Why It Matters
Three-pointer technique	<code>prev</code> , <code>curr</code> , <code>next</code>	Core iterative reversal pattern
Recursive reversal	Reverse rest, then point <code>curr->next->next = curr</code>	Elegant, uses call stack
Head becomes tail	After reversal, original head points to NULL	Must update tail's next
Tail becomes head	After reversal, original tail is the new head	Must return new head

6. Step-by-Step Algorithm

Iterative

1. Initialize `prev = nullptr`, `curr = head`.
2. While `curr` is not null: a. Save `next = curr->next`. b. Reverse: `curr->next = prev`. c. Move `prev` to `curr`. d. Move `curr` to `next`.
3. Return `prev` (new head).

Recursive

1. Base case: if `head == nullptr` or `head->next == nullptr`, return `head`.
2. Recursively reverse the rest: `newHead = reverseList(head->next)`.
3. Make the next node point back: `head->next->next = head`.
4. Make current node's next null: `head->next = nullptr`.
5. Return `newHead`.

7. Dry Run

Input: 1 → 2 → 3 → 4 → NULL

Iterative Dry Run

Step	prev	curr	next	curr->next	List after step
Init	NULL	1	—	—	1 → 2 → 3 → 4 → NULL
1	NULL	1	2	1->next = NULL	NULL ← 1, 2 → 3 → 4 → NULL
2	1	2	3	2->next = 1	NULL ← 1 ← 2, 3 → 4 → NULL
3	2	3	4	3->next = 2	NULL ← 1 ← 2 ← 3, 4 → NULL
4	3	4	NULL	4->next = 3	NULL ← 1 ← 2 ← 3 ← 4
End	4	NULL	—	—	4 → 3 → 2 → 1 → NULL

Result: 4 → 3 → 2 → 1 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Iterative reversal
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    Node* next = nullptr;

    while (curr) {
        next = curr->next;    // Save next
        curr->next = prev;    // Reverse pointer
        prev = curr;         // Move prev forward
        curr = next;         // Move curr forward
    }
    return prev; // New head
}

// Recursive reversal
Node* reverseListRecursive(Node* head) {
    if (!head || !head->next) return head;

    Node* newHead = reverseListRecursive(head->next);
```

```

    head->next->next = head;
    head->next = nullptr;
    return newHead;
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL" << endl;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);

    cout << "Original: ";
    printList(head);

    head = reverseList(head);
    cout << "Reversed: ";
    printList(head);

    return 0;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp
    return prev

def reverse_list_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverse_list_recursive(head.next)
    head.next.next = head

```

```

    head.next = None
    return new_head

def print_list(head):
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("NULL")

# Example
head = Node(1)
head.next = Node(2)
head.next.next = Node(3)
head.next.next.next = Node(4)

print("Original: ", end="")
print_list(head)

head = reverse_list(head)
print("Reversed: ", end="")
print_list(head)

```

10. Code Explanation

- **Iterative:** Three pointers (`prev`, `curr`, `next`) walk through the list. At each step, `curr->next` is redirected to point to `prev` (the node before it). Then all three pointers move forward by one. The loop ends when `curr` becomes null, and `prev` is the new head.
- **Recursive:** Base case returns when list is empty or has one node. Recursively reverse the rest, then make `head->next->next` point back to `head`, and `head->next = nullptr` to avoid cycles. The recursion unwinds from the end.
- **Key insight:** Both approaches are $O(n)$ time and $O(1)$ space (iterative) or $O(n)$ stack space (recursive).

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(n)$ (call stack)

12. Common Patterns

Pattern	Approach
Reverse entire list	Three-pointer or recursion
Reverse between positions	Traverse to m, reverse from m to n
Reverse in k-groups	Recursively reverse each k-group, connect
Palindrome check	Find middle, reverse second half, compare

13. Common Mistakes

- **Losing the next pointer:** Always save `curr->next` before modifying `curr->next`.
- **Not updating head:** The function must return the new head.
- **Cyclic list:** If the list has a cycle, reversal will never terminate.
- **Forgetting base case in recursion:** Must handle `!head || !head->next`.
- **Not setting `head->next = nullptr` in recursion:** Creates a cycle.

14. Edge Cases

- Empty list: Return `nullptr/None`.
- Single node: Return the same node.
- Two nodes: Should reverse correctly.
- List with cycle: Infinite loop — detect cycle first.

15. Variations

Variation	Description
Reverse between positions	Reverse only from <code>m</code> to <code>n</code> (LeetCode 92)
Reverse in k-groups	Reverse groups of <code>k</code> , leave last group if incomplete (LeetCode 25)
Reverse alternate k-nodes	Reverse <code>k</code> , skip <code>k</code> , reverse <code>k</code> ...

16. Related Data Structures

Structure	Connection
Stack	Can push all nodes to stack, then pop to reverse
Doubly linked list	Reverse by swapping <code>prev</code> and <code>next</code> pointers

17. Practice Problems

Easy

- **Reverse Linked List** — LeetCode 206 — Iterative reversal
- **Palindrome Linked List** — LeetCode 234 — Reverse half, compare

Medium

- **Reverse Linked List II** — LeetCode 92 — Reverse between positions
- **Reverse Nodes in k-Group** — LeetCode 25 — Group reversal

Hard

- **Reverse Nodes in Even Length Groups** — LeetCode 2074 — Group-based reversal

18. Interview Explanation

"To reverse a linked list iteratively, I use three pointers: prev, curr, and next. I save the next node, redirect curr->next to prev, then advance all three. At the end, prev is the new head. The recursive version reverses the rest first, then connects the current node — but it uses $O(n)$ stack space. Iterative is preferred for production code."

19. Revision Notes

- **Iterative:** `prev = NULL; curr = head; while(curr) { next = curr->next; curr->next = prev; prev = curr; curr = next; } return prev;`
- **Recursive:** `newHead = reverse(head->next); head->next->next = head; head->next = NULL; return newHead;`
- Always return the new head.
- Save `next` before overwriting `curr->next`.

20. Final Cheat Sheet

REVERSE LINKED LIST

When to use: Reverse entire list or sublist

Approach: Three-pointer iterative ($O(1)$ space) or recursive ($O(n)$ stack)

Key code: `curr->next = prev; prev = curr; curr = next;`

Edge cases: Empty list, single node, list with cycle

Traps: Losing next pointer, not returning new head, stack overflow in recursion

FAST AND SLOW POINTERS

1. Overview

The **fast and slow pointers** technique (also called Floyd's Tortoise and Hare) uses two pointers that traverse the linked list at different speeds. The **slow** pointer moves one step at a time, while the **fast** pointer moves two steps at a time. This seemingly simple idea solves many linked list problems elegantly.

2. Intuition

Imagine two runners on a circular track. One runs twice as fast as the other. If they start together, the faster runner will lap the slower one — they'll meet again at some point.

- **Why it works:** The relative speed between the two pointers is 1 step per iteration. If there's a cycle, the fast pointer will eventually catch up to the slow pointer from behind (in terms of the cycle).
- **Middle finding:** When the fast pointer reaches the end (2x speed), the slow pointer has covered exactly half the distance — it's at the middle.

3. When to Use It

- **Detect cycle** in a linked list.
- **Find the middle** of a linked list.

- **Find the start of a cycle.**
- **Find the nth node from the end** (move one pointer ahead by n, then move both).
- **Check if a linked list is a palindrome** (find middle, reverse second half).
- **Find the intersection point** of two linked lists.

Trigger phrases:

- "Cycle detection"
- "Middle of linked list"
- "Nth from end"
- "Tortoise and hare"
- "Circular linked list"

4. When Not to Use It

- For static arrays — use `arr[mid]` directly.
- If the list is empty or has 1 element — handle as edge case.
- If you need to detect a cycle in a graph (use DFS/visited set).
- If the list is extremely long and you need exact position — two-pointer still works but may need extra logic.

5. Core Concepts

Concept	Explanation	Why It Matters
Slow pointer	Moves 1 step per iteration	Tracks "half" distance
Fast pointer	Moves 2 steps per iteration	Covers distance 2x faster
Relative speed	1 step per iteration	Fast catches slow in cycles
Meeting point	Where <code>slow == fast</code> in a cycle	Proves cycle exists
Cycle entry	Reset one pointer to head, both at 1x speed, meets at cycle start	Find where cycle begins

6. Step-by-Step Algorithm

Detect Cycle

1. Initialize `slow = head, fast = head`.
2. While `fast` and `fast->next` are not null: a. `slow = slow->next` (1 step). b. `fast = fast->next->next` (2 steps). c. If `slow == fast`, cycle exists. Return true.
3. Return false (no cycle).

Find Middle

1. `slow = head, fast = head`.
2. While `fast` and `fast->next` are not null: a. `slow = slow->next`. b. `fast = fast->next->next`.

3. Return `slow` (middle node).

Find Start of Cycle

1. Detect cycle using Floyd's algo. If no cycle, return null.
2. Reset `slow = head`.
3. Move both `slow` and `fast` at 1x speed.
4. When they meet again, that node is the cycle start.

7. Dry Run

Cycle detection: 1 → 2 → 3 → 4 → 5 → 3 (cycle)

Step	slow	fast	Condition
0	1	1	Start
1	2	3	—
2	3	5	—
3	4	4	Meet! Cycle detected

Find middle: 1 → 2 → 3 → 4 → 5 → NULL

Step	slow	fast
0	1	1
1	2	3
2	3	5
3	—	fast->next == NULL, stop

Middle = 3 (for even length, returns second middle or first depending on exact condition)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Detect cycle
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast && fast->next) {
```

```
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

// Find middle (returns second middle for even length)
Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// Find start of cycle
Node* detectCycleStart(Node* head) {
    Node* slow = head;
    Node* fast = head;
    bool hasCycle = false;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            hasCycle = true;
            break;
        }
    }

    if (!hasCycle) return nullptr;

    slow = head;
    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}

// Find nth node from end
Node* nthFromEnd(Node* head, int n) {
    Node* slow = head;
    Node* fast = head;

    for (int i = 0; i < n; i++) {
        if (!fast) return nullptr; // n > length
        fast = fast->next;
    }

    while (fast) {
```

```
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

9. Python Implementation

```
class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False

def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

def detect_cycle_start(head):
    slow = fast = head
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break
    if not has_cycle:
        return None
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next
    return slow

def nth_from_end(head, n):
    slow = fast = head
    for _ in range(n):
        if not fast:
            return None
```

```
    fast = fast.next
while fast:
    slow = slow.next
    fast = fast.next
return slow
```

10. Code Explanation

- **Cycle detection:** The fast pointer moves 2x. If there's a cycle, it will eventually lap the slow pointer and they'll meet. The while loop condition `fast && fast->next` prevents null pointer access when the list has no cycle.
- **Middle finding:** When fast reaches the end (or can't take 2 steps), slow is at the middle. For even-length lists, `slow` ends up at the second middle element.
- **Cycle start:** After detecting a cycle, reset one pointer to head. Move both at 1x. They meet at the cycle start. This works because the distance from head to cycle start equals the distance from meeting point to cycle start (in the cycle).
- **nth from end:** Move fast ahead by n, then move both together. When fast reaches end, slow is at the nth node from end.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Detect cycle	$O(n)$	$O(1)$
Find middle	$O(n)$	$O(1)$
Find cycle start	$O(n)$	$O(1)$
nth from end	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Cycle detection	Floyd's algorithm
Middle of list	Classic slow/fast
Cycle start	Meet point → reset → 1x each
nth from end	Fast ahead by n, then both 1x
Palindrome	Middle → reverse second half → compare
Happy number	Detect cycle in digit square sum

13. Common Mistakes

- **Not checking `fast->next` for null:** `fast->next->next` will crash if `fast->next` is null.
- **Wrong middle for even lengths:** Some problems want the first middle, some want the second. Clarify.
- **Infinite loop on acyclic list:** Without the null check, the while loop may never terminate.

- **Not handling empty list:** `slow = head` with null head will crash on `slow->next`.

14. Edge Cases

- Empty list: Return null/false.
- Single node: No cycle, middle is the node itself.
- Two nodes: Middle is second node (for standard algorithm).
- Full cycle: Head points to itself — fast catches slow immediately.
- Cycle at tail: Fast catches slow after traversing the cycle.

15. Variations

Variation	Description
Floyd's algorithm	Standard cycle detection
Brent's algorithm	Faster cycle detection, uses less memory access
Three-pointer	For problems like remove nth from end

16. Related Data Structures

Structure	Connection
Hash set	Alternative for cycle detection ($O(n)$ space)
Array	Two-pointer technique also works on sorted arrays

17. Practice Problems

Easy

- **Linked List Cycle** — LeetCode 141 — Cycle detection
- **Middle of the Linked List** — LeetCode 876 — Find middle

Medium

- **Linked List Cycle II** — LeetCode 142 — Find cycle start
- **Remove Nth Node From End** — LeetCode 19 — nth from end

Hard

- **Find the Duplicate Number** — LeetCode 287 — Cycle detection on array indices

18. Interview Explanation

"Fast and slow pointers, also known as Floyd's Tortoise and Hare, uses two pointers moving at different speeds — usually 1x and 2x. It's perfect for cycle detection, finding the middle of a list, and finding the nth node from the end. The key insight is that the relative speed difference lets us detect cycles or find the midpoint in one pass, using $O(1)$ extra space."

19. Revision Notes

- **Cycle:** `slow = slow->next; fast = fast->next->next; if(slow == fast) return true;`
- **Middle:** `slow = slow->next; fast = fast->next->next;` — when fast reaches end, slow is middle.
- **Cycle start:** After meeting, reset `slow = head`, move both 1x until they meet.
- **nth from end:** Move fast ahead by n, then move both 1x until fast is null.
- Always check `fast && fast->next` in while condition.

20. Final Cheat Sheet

FAST AND SLOW POINTERS

When to use: Cycle detection, middle finding, nth from end

Core pattern: `slow += 1 step, fast += 2 steps` per iteration

```
Key code: while(fast && fast->next) { slow = slow->next; fast = fast->next->next; }
```

Edge cases: Empty list, single node, even-length list

Traps: Null pointer on `fast->next`, wrong middle for even length

DETECT CYCLE

1. Overview

Cycle detection in a linked list determines whether there's a loop — a node whose `next` pointer points back to a previous node, creating an infinite cycle. The most famous algorithm is **Floyd's Cycle Detection** (Tortoise and Hare).

2. Intuition

Imagine two runners on a circular track. One runs at 1x speed, the other at 2x speed. If the track is circular, the faster runner will eventually lap the slower one. On a straight track, the faster runner simply reaches the end first.

- **Why it works:** If there's a cycle, the fast pointer (2x) will eventually catch up to the slow pointer (1x) from behind because they're moving in a loop. The relative speed is 1 step per iteration, so the distance between them decreases by 1 each step.

3. When to Use It

- Explicitly asked: "Detect if linked list has a cycle."
- As part of another problem: cycle detection is needed before finding the cycle start, or before reversing/palindrome checking.
- Finding duplicate numbers in an array (using index as pointer).
- Checking if a number is a "happy number."

Trigger phrases:

- "Cycle in linked list"
- "Circular linked list"
- "Loop in list"
- "Floyd's cycle detection"

4. When Not to Use It

- If the list is acyclic and you know it (waste of time).
- If you need to detect cycles in a graph (use DFS + visited set).
- If memory is very tight and you need the simplest possible check (hash set is simpler to code).

5. Core Concepts

Concept	Explanation	Why It Matters
Cycle	A node's <code>next</code> points to a previous node	Creates infinite loop on traversal
Meeting point	Where <code>slow == fast</code> inside the cycle	Proves existence of cycle
Cycle entry	First node of the cycle	Needed for cycle removal
Cycle length	Number of nodes in the cycle	Used in some problems

6. Step-by-Step Algorithm

Floyd's Cycle Detection

1. Initialize `slow = head`, `fast = head`.
2. While `fast` and `fast->next` are not null: a. `slow = slow->next`. b. `fast = fast->next->next`. c. If `slow == fast`, return true (cycle exists).
3. Return false (no cycle).

Find Cycle Start

1. Run Floyd's detection. If no cycle, return null.
2. Reset `slow = head`.
3. Move both `slow` and `fast` at 1x speed.
4. When they meet, that node is the cycle start.

Find Cycle Length

1. After detecting cycle, keep `fast` at meeting point.
2. Move `fast` one step at a time, counting steps, until it returns to meeting point.

7. Dry Run

1 → 2 → 3 → 4 → 5 → 3 (cycle back to 3)

Step	slow	fast	Meeting?
0	1	1	Start

Step	slow	fast	Meeting?
1	2	3	No
2	3	5	No
3	4	4 (3's next)	Yes

Find cycle start: After meeting at 4, reset `slow = 1`. Move both at 1x:

- slow: 1, fast: 5
- slow: 2, fast: 3
- slow: 3, fast: 4
- slow: 4, fast: 5
- slow: 5, fast: 3
- slow: 3, fast: 3 → **Meet at 3, which is cycle start**

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Detect cycle
bool hasCycle(Node* head) {
    if (!head) return false;
    Node* slow = head;
    Node* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

// Find cycle start
Node* detectCycleStart(Node* head) {
    if (!head) return nullptr;
    Node* slow = head;
    Node* fast = head;
    bool hasCycle = false;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            hasCycle = true;

```

```

        break;
    }
}
if (!hasCycle) return nullptr;
slow = head;
while (slow != fast) {
    slow = slow->next;
    fast = fast->next;
}
return slow;
}

// Find cycle length
int cycleLength(Node* head) {
    Node* slow = head;
    Node* fast = head;
    bool hasCycle = false;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            hasCycle = true;
            break;
        }
    }
    if (!hasCycle) return 0;
    int len = 1;
    fast = fast->next;
    while (fast != slow) {
        fast = fast->next;
        len++;
    }
    return len;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def has_cycle(head):
    if not head:
        return False
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True

```

```
    return False

def detect_cycle_start(head):
    if not head:
        return None
    slow = fast = head
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break
    if not has_cycle:
        return None
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next
    return slow

def cycle_length(head):
    slow = fast = head
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break
    if not has_cycle:
        return 0
    length = 1
    fast = fast.next
    while fast != slow:
        fast = fast.next
        length += 1
    return length
```

10. Code Explanation

- **hasCycle:** Standard Floyd's algorithm. The while loop condition `fast && fast->next` ensures we don't dereference null when the list is acyclic.
- **detectCycleStart:** After detecting the cycle, reset `slow` to head. Move both at 1x. The mathematical proof: the distance from head to cycle start = distance from meeting point to cycle start (going around the cycle).
- **cycleLength:** After the meeting point, keep one pointer stationary and move the other around the cycle, counting steps until they meet again.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Detect cycle	O(n)	O(1)
Find cycle start	O(n)	O(1)
Find cycle length	O(n)	O(1)

12. Common Patterns

Pattern	Approach
Floyd's detection	Two pointers, 1x and 2x
Hash set	Store visited nodes, O(n) space
Mark visited	Modify node data (not recommended)

13. Common Mistakes

- **Not checking `fast->next` for null**: Causes segmentation fault.
- **Infinite loop on acyclic list**: Without proper null check.
- **Wrong cycle start**: The mathematical proof requires resetting to head, not to the meeting point.
- **Not handling empty list**: `head == nullptr` should return false/null.

14. Edge Cases

- Empty list: No cycle.
- Single node with `next = nullptr`: No cycle.
- Single node with `next = itself`: Cycle of length 1.
- Full cycle: Head points to itself or tail points to head.
- Cycle at the very end: Tail points to some middle node.

15. Variations

Variation	Description
Brent's algorithm	Moves fast pointer at 2^k steps, then resets; fewer pointer dereferences
Hash set approach	O(n) space, simpler to code, guaranteed to find cycle

16. Related Data Structures

Structure	Connection
Hash set	Alternative detection method, O(n) space
Graph	Cycle detection in directed/undirected graphs uses DFS

17. Practice Problems

Easy

- **Linked List Cycle** — LeetCode 141 — Detect cycle
- **Happy Number** — LeetCode 202 — Cycle detection on digit squares

Medium

- **Linked List Cycle II** — LeetCode 142 — Find cycle start
- **Find the Duplicate Number** — LeetCode 287 — Cycle detection on array

Hard

- **Circular Array Loop** — LeetCode 457 — Detect cycle in array with direction

18. Interview Explanation

"I use Floyd's Cycle Detection algorithm — two pointers moving at 1x and 2x speeds. If they meet, there's a cycle. To find the cycle start, I reset one pointer to head and move both at 1x until they meet again. The algorithm is $O(n)$ time and $O(1)$ space. I also know the hash set approach for when space isn't a concern."

19. Revision Notes

- Floyd's: `slow = slow->next; fast = fast->next->next; if(slow == fast) → cycle.`
- Cycle start: Reset slow to head, move both 1x until they meet.
- Cycle length: After meeting, move one pointer around the cycle, counting steps.
- Always check `fast && fast->next` in while condition.
- $O(n)$ time, $O(1)$ space.

20. Final Cheat Sheet

DETECT CYCLE

When to use: Any linked list cycle detection

Algorithm: Floyd's Tortoise and Hare

Key code: `while(fast && fast->next) { slow = slow->next; fast = fast->next->next; if(slow == fast) return true; }`

Cycle start: Reset slow to head, move both 1x until meet

Edge cases: Empty list, single node, self-loop

Traps: Null pointer on fast->next, not checking fast itself

FIND MIDDLE

1. Overview

Find the middle node of a linked list. For a list with an odd number of nodes, return the middle. For an even number of nodes, return the second middle (or the first, depending on the problem).

2. Intuition

Use the slow/fast pointer technique. When the fast pointer (2x speed) reaches the end, the slow pointer (1x speed) is at the middle. It's like two cars racing — when the faster car finishes the track, the slower car is exactly halfway.

- **Why it works:** The fast pointer covers twice the distance in the same time. So when fast covers n steps, slow covers $n/2$ steps — exactly the middle.

3. When to Use It

- **Palindrome check:** Find middle, reverse second half, compare.
- **Binary search on linked list:** Find middle, divide.
- **Merge sort on linked list:** Find middle to split the list.
- **Reorder list:** Find middle, then interleave halves.
- **Delete middle node.**

Trigger phrases:

- "Middle of linked list"
- "Split linked list into two halves"
- "Palindrome linked list"
- "Reorder list"

4. When Not to Use It

- If you have the length of the list, just do $\text{length}/2$ steps from head.
- If you need both middle nodes for even-length list, adjust the algorithm (use `fast` starts at `head->next`).

5. Core Concepts

Concept	Explanation	Why It Matters
Odd length	Middle is unique	Slow pointer returns correct node
Even length	Two middles exist	Problem may ask for first or second middle
Second middle	Position at $n/2$ (0-indexed)	Default for <code>while(fast && fast->next)</code>
First middle	Position at $(n-1)/2$	Needs <code>fast = head->next</code> initialization

6. Step-by-Step Algorithm

Standard (returns second middle for even length)

1. Initialize `slow = head, fast = head`.
2. While `fast` and `fast->next` are not null: a. `slow = slow->next`. b. `fast = fast->next->next`.
3. Return `slow`.

First middle for even length

1. Initialize `slow = head, fast = head->next`.

- While `fast` and `fast->next` are not null: a. `slow = slow->next`. b. `fast = fast->next->next`.
- Return `slow`.

7. Dry Run

1 → 2 → 3 → 4 → 5 (odd, n=5, middle=3)

Step	slow	fast	Condition
0	1	1	Start
1	2	3	—
2	3	5	fast->next == null, stop

Result: 3

1 → 2 → 3 → 4 (even, n=4)

Step	slow	fast	Condition
0	1	1	Start
1	2	3	—
2	3	null	fast == null, stop

Result: 3 (second middle)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Second middle (default)
Node* findMiddle(Node* head) {
    if (!head) return nullptr;
    Node* slow = head;
    Node* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// First middle (for even length)
```

```
Node* findMiddleFirst(Node* head) {
    if (!head) return nullptr;
    Node* slow = head;
    Node* fast = head->next;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

9. Python Implementation

```
class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def find_middle(head):
    if not head:
        return None
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

def find_middle_first(head):
    if not head:
        return None
    slow = head
    fast = head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

10. Code Explanation

- **Standard approach:** `slow` moves 1 step, `fast` moves 2 steps. When `fast` reaches the end (or its next is null), `slow` is at the middle.
- **First middle:** By initializing `fast = head->next`, the slow pointer lags by an extra half step, so it ends up at the first middle for even-length lists.
- **Edge case:** Empty list returns null immediately.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Standard (second middle)	$O(n)$	$O(1)$
First middle variant	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Palindrome check	Middle → reverse second half → compare
Merge sort	Middle → split → recurse → merge
Reorder list	Middle → reverse second half → interleave
Delete middle	Find middle node, then delete it

13. Common Mistakes

- **Wrong middle for even length:** Know which middle the problem expects.
- **Not handling empty list:** Returns null.
- **Not handling single node:** Middle is the node itself.
- **Off-by-one in odd-length:** The algorithm naturally handles this.

14. Edge Cases

- Empty list: Return null.
- Single node: Return that node.
- Two nodes: Standard returns second node, first-middle variant returns first node.
- Large list: Works correctly, no overflow.

15. Variations

Variation	Description
First middle	For even lengths, return the first of the two middles
Second middle	Default, return the second of the two middles

16. Related Data Structures

Structure	Connection
Array	<code>arr[arr.length/2]</code> is $O(1)$ vs $O(n)$ for linked list
Fast/slow pointers	Same technique, different use case

17. Practice Problems

Easy

- **Middle of the Linked List** — LeetCode 876 — Find middle
- **Delete the Middle Node of a Linked List** — LeetCode 2095 — Find and delete middle

Medium

- **Palindrome Linked List** — LeetCode 234 — Middle + reverse
- **Reorder List** — LeetCode 143 — Middle + reverse + interleave

Hard

- **Merge Sort on Linked List** — LeetCode 148 — Middle + split + merge

18. Interview Explanation

"To find the middle of a linked list, I use the slow and fast pointer technique. Slow moves 1 step, fast moves 2 steps. When fast reaches the end, slow is at the middle. For even-length lists, I clarify which middle is expected — second middle is the default. It's $O(n)$ time and $O(1)$ space."

19. Revision Notes

- Standard: `slow = slow->next; fast = fast->next->next;` while `fast && fast->next`.
- Second middle for even length (default).
- First middle: `fast = head->next` initially.
- $O(n)$ time, $O(1)$ space.
- Used in palindrome, merge sort, reorder list.

20. Final Cheat Sheet

FIND MIDDLE

When to use: Split list, palindrome check, merge sort

Algorithm: Slow/fast pointer (1x and 2x)

Key code: `while(fast && fast->next) { slow = slow->next; fast = fast->next->next; }`

Edge cases: Empty list, single node, even length

Traps: Wrong middle for even length, null pointer

MERGE TWO SORTED LISTS

1. Overview

Merge two sorted linked lists into one sorted linked list. Given the heads of two sorted lists, combine them into a single sorted list by rearranging pointers.

2. Intuition

Think of merging two sorted decks of cards. You look at the top card of each deck, pick the smaller one, and place it on the output pile. Repeat until both decks are empty.

- **Why it works:** At each step, we pick the smallest remaining element from the two lists. Since both lists are sorted, the smallest remaining element is always at one of the two heads. This is a two-pointer merge, similar to the merge step in merge sort.

3. When to Use It

- Given two sorted linked lists, merge them into one sorted list.
- As part of **merge sort on linked list** (split → sort → merge).
- Combining multiple sorted lists (k-way merge with heap).
- As a subroutine in more complex list operations.

Trigger phrases:

- "Merge two sorted lists"
- "Merge sorted linked lists"
- "Combine two sorted lists"
- "Merge sort on linked list"

4. When Not to Use It

- If the lists are not sorted (sort them first, or use a different approach).
- If you need to merge more than two lists at once, use a min-heap (k-way merge).
- If you need to merge arrays, the same approach works but with indices instead of pointers.

5. Core Concepts

Concept	Explanation	Why It Matters
Dummy node	Placeholder node before the result head	Simplifies edge cases, avoids null checks
Tail pointer	Points to the last node of the merged list	O(1) insertion at end
Two-pointer merge	Compare heads of both lists, pick smaller	Core merge logic
Remaining elements	One list may be exhausted before the other	Append remaining nodes directly

6. Step-by-Step Algorithm

1. Create a dummy node (value doesn't matter). Set `tail = dummy`.
2. While both lists are non-empty: a. Compare `l1->data` and `l2->data`. b. Attach the smaller node to `tail->next`. c. Move the pointer of the list we took from. d. Move `tail = tail->next`.
3. If one list is non-empty, attach the rest to `tail->next`.
4. Return `dummy->next` (the actual head of the merged list).

7. Dry Run

List 1: 1 → 3 → 5 List 2: 2 → 4 → 6

Step	l1	l2	tail->next	Merged list so far
0	1	2	dummy	[]
1	3	2	1	1 →
2	3	4	2	1 → 2 →
3	5	4	3	1 → 2 → 3 →
4	5	6	4	1 → 2 → 3 → 4 →
5	null	6	5	1 → 2 → 3 → 4 → 5 →
6	null	null	6	1 → 2 → 3 → 4 → 5 → 6 → NULL

Result: 1 → 2 → 3 → 4 → 5 → 6 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Iterative merge
Node* mergeTwoLists(Node* l1, Node* l2) {
    Node dummy(0);
    Node* tail = &dummy;

    while (l1 && l2) {
        if (l1->data <= l2->data) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }

    tail->next = l1 ? l1 : l2;
    return dummy.next;
}

// Recursive merge
Node* mergeTwoListsRecursive(Node* l1, Node* l2) {
    if (!l1) return l2;
```

```

    if (!l2) return l1;

    if (l1->data <= l2->data) {
        l1->next = mergeTwoListsRecursive(l1->next, l2);
        return l1;
    } else {
        l2->next = mergeTwoListsRecursive(l1, l2->next);
        return l2;
    }
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL" << endl;
}

int main() {
    Node* l1 = new Node(1);
    l1->next = new Node(3);
    l1->next->next = new Node(5);

    Node* l2 = new Node(2);
    l2->next = new Node(4);
    l2->next->next = new Node(6);

    Node* merged = mergeTwoLists(l1, l2);
    printList(merged); // 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL

    return 0;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def merge_two_lists(l1, l2):
    dummy = Node(0)
    tail = dummy

    while l1 and l2:
        if l1.data <= l2.data:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2

```

```

        l2 = l2.next
        tail = tail.next

    tail.next = l1 if l1 else l2
    return dummy.next

def merge_two_lists_recursive(l1, l2):
    if not l1:
        return l2
    if not l2:
        return l1

    if l1.data <= l2.data:
        l1.next = merge_two_lists_recursive(l1.next, l2)
        return l1
    else:
        l2.next = merge_two_lists_recursive(l1, l2.next)
        return l2

def print_list(head):
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("NULL")

# Example
l1 = Node(1)
l1.next = Node(3)
l1.next.next = Node(5)

l2 = Node(2)
l2.next = Node(4)
l2.next.next = Node(6)

merged = merge_two_lists(l1, l2)
print_list(merged)  # 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL

```

10. Code Explanation

- **Dummy node:** A stack-allocated dummy node (`Node dummy(0)`) avoids null checks. We return `dummy.next` as the result.
- **While loop:** Compares the current heads of both lists. The smaller node is appended to the merged list. The pointer of that list advances.
- **Remaining nodes:** After the loop, one list may be non-empty. We append it directly — no need to copy, just connect the pointer.
- **Recursive version:** Base case returns the other list if one is null. Otherwise, recursively merge the rest.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Iterative	$O(n + m)$	$O(1)$
Recursive	$O(n + m)$	$O(n + m)$ (call stack)

Where n and m are the lengths of the two lists.

12. Common Patterns

Pattern	Approach
Two-pointer merge	Compare and advance
Dummy node	Simplify head handling
Recursive merge	Elegant but uses stack space
K-way merge	Use min-heap for k lists

13. Common Mistakes

- **Forgetting to advance the tail pointer:** `tail = tail->next` after each append.
- **Not handling empty lists:** One or both lists may be null.
- **Creating new nodes:** The problem usually asks to rearrange existing nodes, not create new ones.
- **Losing the original heads:** Use dummy node to avoid losing the result head.

14. Edge Cases

- Both lists empty: Return null.
- One list empty: Return the other list.
- Lists of different lengths: Handled by the remaining-nodes step.
- Duplicate values: Use `<=` for stable merge (preserves order of equal elements).

15. Variations

Variation	Description
Merge k sorted lists	Use min-heap (priority queue)
Merge sort on linked list	Recursively split, sort, and merge
In-place merge	Rearrange pointers without extra nodes

16. Related Data Structures

Structure	Connection
Min-heap	For merging k sorted lists
Array	Same merge algorithm, different indexing

17. Practice Problems

Easy

- **Merge Two Sorted Lists** — LeetCode 21 — Standard merge
- **Merge Sorted Array** — LeetCode 88 — Similar concept on arrays

Medium

- **Merge k Sorted Lists** — LeetCode 23 — Min-heap or divide & conquer
- **Sort List** — LeetCode 148 — Merge sort on linked list

Hard

- **Merge k Sorted Lists (divide & conquer)** — LeetCode 23 — $O(n \log k)$ solution

18. Interview Explanation

"I use a dummy node to simplify the merge. I compare the heads of both lists, attach the smaller node to the tail of the result, and advance the corresponding pointer. When one list is exhausted, I append the remaining nodes of the other list. The time complexity is $O(n+m)$ and space is $O(1)$ for the iterative approach."

19. Revision Notes

- Dummy node: `Node dummy(0); Node* tail = &dummy;`
- While both non-empty: compare, attach smaller, advance.
- Append remaining: `tail->next = l1 ? l1 : l2;`
- Return `dummy.next`.
- Recursive: `if(!l1) return l2; if(!l2) return l1;`
- $O(n+m)$ time, $O(1)$ space (iterative).

20. Final Cheat Sheet

MERGE TWO SORTED LISTS

When to use: Merging two sorted lists into one sorted list

Algorithm: Two-pointer compare-and-append

Key code: `dummy(0); tail = &dummy; while(l1 && l2) { ... } tail->next = l1 ? l1 : l2;`

Edge cases: One or both lists empty

Traps: Forgetting to advance tail, creating new nodes instead of rearranging

REMOVE NTH NODE FROM END

1. Overview

Remove the nth node from the end of a linked list in one pass. Given a linked list and an integer `n`, delete the `n`th node from the end and return the head.

2. Intuition

Use two pointers with an offset. Move the fast pointer ahead by `n` steps, then move both pointers together until fast reaches the end. The slow pointer is now at the node just before the target node.

- **Why it works:** When fast starts `n` steps ahead and both move at the same speed, the distance between them remains `n`. When fast reaches the end (null), slow is `n` steps behind — exactly at the node before the target.

3. When to Use It

- Explicitly asked: "Remove nth node from end."
- Deleting the last `k` nodes in $O(n)$.
- Finding the `n`th node from the end (same technique, just don't delete).

Trigger phrases:

- "Remove nth node from end"
- "Delete from end"
- "Kth from end"
- "Remove last k elements"

4. When Not to Use It

- If you know the length of the list, you can just do `length - n` steps from head.
- If `n` is guaranteed to be 1 (delete last node), just traverse to the second-to-last node.
- If the list is very short (`n > length`), handle gracefully.

5. Core Concepts

Concept	Explanation	Why It Matters
Offset technique	Fast pointer starts <code>n</code> steps ahead	Creates the gap needed to find the target
Dummy node	Placeholder before head	Simplifies case when head is deleted
Node before target	Slow pointer stops at $(n+1)$ th from end	Need previous node to update next pointer

6. Step-by-Step Algorithm

1. Create a dummy node. Set `dummy->next = head`.
2. Initialize `slow = fast = dummy`.
3. Move `fast` ahead by `n + 1` steps (so slow is at the node before the target).
4. While `fast` is not null: a. `slow = slow->next`. b. `fast = fast->next`.
5. Now `slow->next` is the node to delete.
6. Set `slow->next = slow->next->next`.
7. Return `dummy->next` (new head).

7. Dry Run

List: 1 → 2 → 3 → 4 → 5, n = 2 (remove 4)

Step	slow	fast	Action
0	dummy	dummy	Start
1	dummy	dummy	Move fast ahead by n+1=3
2	dummy	2	—
3	dummy	3	—
4	dummy	4	Fast moved 3 steps
5	1	5	Both move
6	2	null	Fast reached end, stop
7	—	—	<code>slow->next = slow->next->next</code> (delete 4)

Result: 1 → 2 → 3 → 5 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

Node* removeNthFromEnd(Node* head, int n) {
    Node dummy(0);
    dummy.next = head;
    Node* slow = &dummy;
    Node* fast = &dummy;

    // Move fast ahead by n+1
    for (int i = 0; i <= n; i++) {
        if (!fast) return head; // n > length
        fast = fast->next;
    }

    // Move both until fast reaches end
    while (fast) {
        slow = slow->next;
        fast = fast->next;
    }

    // Delete the target node
```

```

    Node* temp = slow->next;
    slow->next = slow->next->next;
    delete temp;

    return dummy.next;
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL" << endl;
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    cout << "Original: ";
    printList(head);

    head = removeNthFromEnd(head, 2);
    cout << "After removing 2nd from end: ";
    printList(head);

    return 0;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def remove_nth_from_end(head, n):
    dummy = Node(0)
    dummy.next = head
    slow = fast = dummy

    # Move fast ahead by n+1
    for _ in range(n + 1):
        if not fast:
            return head
        fast = fast.next

    # Move both until fast reaches end

```

```

while fast:
    slow = slow.next
    fast = fast.next

# Delete the target node
slow.next = slow.next.next

return dummy.next

def print_list(head):
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("NULL")

# Example
head = Node(1)
head.next = Node(2)
head.next.next = Node(3)
head.next.next.next = Node(4)
head.next.next.next.next = Node(5)

print("Original: ", end="")
print_list(head)

head = remove_nth_from_end(head, 2)
print("After removing 2nd from end: ", end="")
print_list(head)

```

10. Code Explanation

- **Dummy node:** Critical for the case where $n = \text{length}$ (delete head). Without it, deleting the head would be a special case.
- **Fast offset:** `fast` moves $n+1$ steps ahead so that `slow` ends up at the node **before** the target. If we moved n steps, `slow` would be at the target node itself, and we couldn't delete it without extra pointer.
- **Delete:** `slow->next = slow->next->next` bypasses the target node. In C++, we also free the memory with `delete`.
- **Edge case:** If $n > \text{length}$, the function returns the original head without modification.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Remove nth from end	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Two-pointer with offset	Fast ahead by n , then both move

Pattern	Approach
Dummy node	Handles head deletion
Find nth from end	Same technique, just read instead of delete

13. Common Mistakes

- **Moving fast by n instead of n+1:** Slow will point to the target node, not the node before it.
- **Not using dummy node:** Head deletion becomes a special case.
- **Off-by-one in offset:** Moving `n` steps vs `n+1` steps.
- **Memory leak in C++:** Forgetting to `delete` the removed node.
- **Not handling `n > length`:** Should return head without modification.

14. Edge Cases

- `n = 1` (delete last node): Works correctly.
- `n = length` (delete head): Dummy node handles this.
- `n > length`: Return head unchanged.
- Single node, `n = 1`: Return null.
- Empty list: Return null.

15. Variations

Variation	Description
Find nth from end	Same technique, return <code>slow->next</code> instead of deleting
Delete middle node	Find middle, then delete
Delete kth from end in a doubly linked list	Move from tail backward

16. Related Data Structures

Structure	Connection
Doubly linked list	Can also remove from end using <code>tail->prev</code> ($O(1)$ if tail known)
Array	<code>arr[arr.length - n]</code> is $O(1)$ vs $O(n)$ for linked list

17. Practice Problems

Easy

- **Remove Nth Node From End of List** — LeetCode 19 — Standard problem
- **Delete Node in a Linked List** — LeetCode 237 — Given only the node, not head

Medium

- **Remove Duplicates from Sorted List II** — LeetCode 82 — Two-pointer with prev
- **Remove Linked List Elements** — LeetCode 203 — Remove all occurrences

Hard

- **Remove Zero Sum Consecutive Nodes from Linked List** — LeetCode 1171 — Prefix sum + hashmap

18. Interview Explanation

"I use two pointers with a dummy node. I move the fast pointer $n+1$ steps ahead, then move both pointers together until fast reaches the end. The slow pointer is now at the node before the target, so I can delete it in $O(1)$. The dummy node handles the edge case where the head is deleted. Time is $O(n)$, space is $O(1)$."

19. Revision Notes

- Dummy node: `Node dummy(0); dummy.next = head; slow = &dummy; fast = &dummy;`
- Move fast ahead by $n+1$ steps.
- Move both until `fast == null`.
- Delete: `slow->next = slow->next->next`.
- Return `dummy.next`.

20. Final Cheat Sheet

REMOVE NTH NODE FROM END

When to use: Delete nth node from end of linked list

Algorithm: Two-pointer with $n+1$ offset

Key code: `fast = dummy; for(int i=0; i<=n; i++) fast = fast->next; while(fast) { slow=slow->next; fast=fast->next; }`

Edge cases: $n=1$, $n=length$, $n > length$, single node

Traps: Off-by-one in offset, no dummy node, memory leak

INTERSECTION OF LINKED LISTS

1. Overview

Given two singly linked lists, find the node where they intersect (if any). Intersection means the two lists share a common suffix — they merge into one list at some node.

2. Intuition

Think of two roads that merge into one. After the merge point, the roads are the same. The intersection node is the first common node.

- **Why it works (two-pointer approach):** If we traverse both lists, the pointers will eventually meet at the intersection. By switching pointers when they reach the end, we effectively balance the difference in lengths.

3. When to Use It

- Explicitly asked: "Find intersection of two linked lists."
- Two lists share a common suffix.
- Detecting if two lists merge.

Trigger phrases:

- "Intersection of two linked lists"
- "First common node"
- "Merge point of two lists"
- "Where do two lists meet"

4. When Not to Use It

- If the lists are guaranteed to be disjoint (no intersection).
- If you need to find intersection by value (not by reference/node identity).
- For arrays, use hash set to find common elements.

5. Core Concepts

Concept	Explanation	Why It Matters
Intersection node	First node where two lists share a reference	Must be the same object, not just same value
Length difference	Two lists may have different lengths before intersection	Need to align pointers
Two-pointer alignment	Traverse both lists, switch at end	Eliminates need to compute lengths

6. Step-by-Step Algorithm

Approach 1: Length-based

1. Compute lengths of both lists.
2. Advance the longer list's pointer by the difference in lengths.
3. Move both pointers together until they meet (or reach end).

Approach 2: Two-pointer switching (elegant)

1. Initialize `p1 = headA`, `p2 = headB`.
2. While `p1 != p2`: a. `p1 = p1 ? p1->next : headB`. b. `p2 = p2 ? p2->next : headA`.
3. Return `p1` (could be null if no intersection).

7. Dry Run

List A: 1 → 2 → 3 → 4 → 5 → 6 **List B:** 7 → 8 → 5 → 6 (intersection at 5)

Two-pointer switching:

Step	p1	p2	Notes
0	1	7	Start
1	2	8	—
2	3	5	—
3	4	6	—
4	5	null	p2 reaches end
5	6	1	p2 switches to headA
6	null	2	p1 reaches end
7	7	3	p1 switches to headB
8	8	4	—
9	5	5	Meet at intersection!

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Approach 1: Length-based
int getLength(Node* head) {
    int len = 0;
    while (head) {
        len++;
        head = head->next;
    }
    return len;
}

Node* getIntersectionNodeLength(Node* headA, Node* headB) {
    int lenA = getLength(headA);
    int lenB = getLength(headB);

    // Align pointers
    while (lenA > lenB) {
        headA = headA->next;
        lenA--;
    }
    while (lenB > lenA) {
        headB = headB->next;
    }
}
```

```

        lenB--;
    }

    // Find intersection
    while (headA && headB) {
        if (headA == headB) return headA;
        headA = headA->next;
        headB = headB->next;
    }
    return nullptr;
}

// Approach 2: Two-pointer switching (elegant)
Node* getIntersectionNode(Node* headA, Node* headB) {
    if (!headA || !headB) return nullptr;

    Node* p1 = headA;
    Node* p2 = headB;

    while (p1 != p2) {
        p1 = p1 ? p1->next : headB;
        p2 = p2 ? p2->next : headA;
    }

    return p1; // Could be null or intersection node
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def get_length(head):
    length = 0
    while head:
        length += 1
        head = head.next
    return length

def get_intersection_node_length(headA, headB):
    lenA = get_length(headA)
    lenB = get_length(headB)

    while lenA > lenB:
        headA = headA.next
        lenA -= 1
    while lenB > lenA:
        headB = headB.next
        lenB -= 1

```

```

while headA and headB:
    if headA is headB:
        return headA
    headA = headA.next
    headB = headB.next
return None

def get_intersection_node(headA, headB):
    if not headA or not headB:
        return None

    p1, p2 = headA, headB

    while p1 is not p2:
        p1 = p1.next if p1 else headB
        p2 = p2.next if p2 else headA

    return p1

```

10. Code Explanation

- **Length-based approach:** Compute lengths, align the longer list's pointer to match the shorter list's start position, then move both together until they match.
- **Two-pointer switching:** The elegant approach. Each pointer traverses both lists. When one pointer reaches the end, it switches to the other list's head. After at most 2 traversals, they meet at the intersection (or both become null).
- **Why switching works:** Both pointers traverse the same total distance ($\text{lenA} + \text{lenB}$). After the first pass, they're aligned at the intersection point. **p1** traverses A's non-intersecting part + intersection + B's non-intersecting part. **p2** traverses B's non-intersecting part + intersection + A's non-intersecting part. They meet at the intersection.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Length-based	$O(n + m)$	$O(1)$
Two-pointer switching	$O(n + m)$	$O(1)$
Hash set	$O(n + m)$	$O(n)$

12. Common Patterns

Pattern	Approach
Length alignment	Compute lengths, align, traverse together
Pointer switching	Elegant, no length computation needed
Hash set	Store visited nodes of one list, check against the other

13. Common Mistakes

- **Comparing by value instead of reference:** Intersection is about node identity, not data equality.
- **Infinite loop in switching:** Without the `if (p1)` check, switching to the other list would loop forever.
- **Not handling no intersection:** Both approaches return null if no intersection exists.
- **Modifying the lists:** The problem expects the lists to remain unchanged.

14. Edge Cases

- No intersection: Return null.
- One list is empty: Return null.
- Both lists are identical from head: The first node is the intersection.
- Intersection at the last node: Works correctly.
- Lists of very different lengths: Handled by both approaches.

15. Variations

Variation	Description
Find intersection by value	Find first node with same value, different algorithm
Multiple merge points	After first merge, lists remain merged (standard)

16. Related Data Structures

Structure	Connection
Hash set	Alternative $O(n)$ space approach
Tree	LCA (lowest common ancestor) is similar concept in trees

17. Practice Problems

Easy

- **Intersection of Two Linked Lists** — LeetCode 160 — Standard problem
- **Middle of the Linked List** — LeetCode 876 — Related traversal technique

Medium

- **Linked List Cycle II** — LeetCode 142 — Similar two-pointer technique
- **Find the Duplicate Number** — LeetCode 287 — Cycle detection on array

Hard

- **Minimum Number of Operations to Sort a Linked List by Frequency** — Similar pointer manipulation

18. Interview Explanation

"I use the two-pointer switching approach. Both pointers traverse both lists — when one reaches the end, it switches to the other list's head. After at most two traversals, they meet at the intersection or

both become null. This is $O(n+m)$ time and $O(1)$ space. The key insight is that both pointers end up traversing the same total distance, which aligns them at the intersection."

19. Revision Notes

- Two-pointer switching: `p1 = p1 ? p1->next : headB; p2 = p2 ? p2->next : headA;`
- Length-based: align by difference, then traverse together.
- Compare by **reference** (`==` in C++, `is` in Python), not by value.
- $O(n+m)$ time, $O(1)$ space.
- No intersection → return null.

20. Final Cheat Sheet

INTERSECTION OF LINKED LISTS

When to use: Two lists share a common suffix

Algorithm: Two-pointer switching or length alignment

Key code: `while(p1 != p2) { p1 = p1 ? p1->next : headB; p2 = p2 ? p2->next : headA; }`

Edge cases: No intersection, one empty list, identical from head

Traps: Comparing by value, infinite loop without null check

PALINDROME LINKED LIST

1. Overview

Check if a singly linked list is a palindrome — reads the same forward and backward. For example, `1 → 2 → 3 → 2 → 1` is a palindrome.

2. Intuition

A palindrome is symmetric. We can check by finding the middle, reversing the second half, and comparing both halves. This is like folding a string in half and checking if both sides match.

- **Why it works:** If the list is a palindrome, the first half and the reversed second half are identical. By reversing the second half, we can compare element by element without extra space.

3. When to Use It

- Explicitly asked: "Check if linked list is palindrome."
- Any problem where symmetry is involved.

Trigger phrases:

- "Palindrome linked list"
- "Symmetric linked list"
- "Check if list is palindrome"

4. When Not to Use It

- If you can use extra space, copying to an array and checking with two pointers is simpler.
- If the list is very long and you can't modify it (but you can always restore it after checking).

5. Core Concepts

Concept	Explanation	Why It Matters
Middle finding	Slow/fast pointer to find the middle	Splits the list into two halves
Reverse second half	Reverse the second half of the list	Allows comparison without extra space
Compare halves	Walk both halves simultaneously	Check if elements match
Restore list	Reverse second half again (optional)	Maintain original list state

6. Step-by-Step Algorithm

1. Find the middle of the list (using slow/fast).
2. Reverse the second half starting from middle.
3. Compare the first half and reversed second half element by element.
4. (Optional) Restore the second half by reversing again.
5. Return true if all elements match, false otherwise.

7. Dry Run

List: 1 → 2 → 3 → 2 → 1 → NULL

Step	Action	State
1	Find middle	Middle = 3
2	Reverse second half	1 → 2 → 3 → 1 → 2 → NULL (after reverse: 3 → 2 → 1, but we mean from 3's next)
2 (correct)	Reverse from middle->next	First: 1 → 2 → 3 → NULL; Second: 1 → 2 → NULL (reversed from 2→1)
3	Compare	1==1, 2==2, end → true

More precisely:

- Original: 1 → 2 → 3 → 2 → 1 → NULL
- Middle: 3 (slow/fast)
- Reverse from 3->next (which is 2 → 1 → NULL): becomes 1 → 2 → NULL
- First half: 1 → 2 → 3 (but we compare only up to middle)
- Compare: 1==1, 2==2, end → palindrome

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

bool isPalindrome(Node* head) {
    if (!head || !head->next) return true;

    // Find middle
    Node* slow = head;
    Node* fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    // slow is at (first middle) for even, middle for odd

    // Reverse second half
    Node* secondHalf = reverseList(slow->next);
    Node* firstHalf = head;
    Node* temp = secondHalf; // Save for restoration

    // Compare
    bool result = true;
    while (secondHalf) {
        if (firstHalf->data != secondHalf->data) {
            result = false;
            break;
        }
        firstHalf = firstHalf->next;
        secondHalf = secondHalf->next;
    }

    // Restore the list (optional but good practice)
    slow->next = reverseList(temp);
}
```

```
    return result;
}
```

9. Python Implementation

```
class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp
    return prev

def is_palindrome(head):
    if not head or not head.next:
        return True

    # Find middle
    slow = fast = head
    while fast.next and fast.next.next:
        slow = slow.next
        fast = fast.next.next

    # Reverse second half
    second_half = reverse_list(slow.next)
    first_half = head
    temp = second_half # Save for restoration

    # Compare
    result = True
    while second_half:
        if first_half.data != second_half.data:
            result = False
            break
        first_half = first_half.next
        second_half = second_half.next

    # Restore the list
    slow.next = reverse_list(temp)

    return result
```

10. Code Explanation

- **Middle finding:** The condition `fast->next && fast->next->next` finds the **first middle** for even-length lists. This ensures the second half starts after the middle.
- **Reverse second half:** `reverseList(slow->next)` where `slow` is the node before the second half.
- **Comparison:** Walk both halves simultaneously. If any mismatch, break and return false.
- **Restoration:** Reverse the second half again to restore the original list. This is optional but good practice for interview questions that say "don't modify the list."

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Find middle	$O(n)$	$O(1)$
Reverse second half	$O(n)$	$O(1)$
Compare	$O(n)$	$O(1)$
Restore	$O(n)$	$O(1)$
Overall	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Middle + reverse + compare	Standard palindrome check
Array copy	Copy to array, use two pointers ($O(n)$ space)
Stack	Push first half to stack, compare with second half

13. Common Mistakes

- **Wrong middle for even-length:** Must use `fast->next->next` condition to get first middle.
- **Not handling odd vs even:** The algorithm naturally handles both with the right middle-finding condition.
- **Not restoring the list:** Some interviewers expect you to restore the original structure.
- **Comparing the entire first half:** The second half may be shorter (for odd-length, the middle node is excluded).

14. Edge Cases

- Empty list: True.
- Single node: True.
- Two nodes (same value): True.
- Two nodes (different values): False.
- Even-length palindrome: `1 2 2 1`.
- Odd-length palindrome: `1 2 3 2 1`.
- Not a palindrome: `1 2 3`.

15. Variations

Variation	Description
Array approach	Copy to array, $O(n)$ space, simpler code
Stack approach	Push first half to stack, pop and compare
Recursive	Use recursion to compare from both ends

16. Related Data Structures

Structure	Connection
Array/String	Two-pointer from ends is simpler for arrays
Stack	Reverses elements for comparison

17. Practice Problems

Easy

- **Palindrome Linked List** — LeetCode 234 — Standard problem
- **Valid Palindrome** — LeetCode 125 — Similar concept on strings

Medium

- **Reverse Linked List II** — LeetCode 92 — Related reversal technique
- **Reorder List** — LeetCode 143 — Middle + reverse + interleave

Hard

- **Palindrome Pairs** — LeetCode 336 — Pair of strings form palindrome

18. Interview Explanation

"I find the middle using slow and fast pointers, reverse the second half, and compare both halves element by element. This is $O(n)$ time and $O(1)$ space. I also restore the list by reversing the second half again. For even-length lists, I use the condition `fast->next->next` to find the first middle, so the comparison works correctly."

19. Revision Notes

- Find middle: `while(fast->next && fast->next->next) { slow = slow->next; fast = fast->next->next; }`
- Reverse second half: `secondHalf = reverse(slow->next);`
- Compare: `while(secondHalf) { if(firstHalf->data != secondHalf->data) return false; ... }`
- Restore: `slow->next = reverse(temp);`
- $O(n)$ time, $O(1)$ space.

20. Final Cheat Sheet

PALINDROME LINKED LIST

When to use: Check if linked list reads same forward and backward

Algorithm: Middle → reverse second half → compare

Key code: `slow->next = reverse(slow->next); while(second) { if(first->data != second->data) return false; }`

Edge cases: Empty list, single node, even/odd length

Traps: Wrong middle for even length, not restoring list

REVERSE NODES IN K-GROUP

1. Overview

Reverse nodes in k-group means dividing the linked list into groups of **k** consecutive nodes and reversing each group. If the last group has fewer than **k** nodes, it is left as is.

Input: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, k = 3

Output: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8

2. Intuition

Think of it as repeatedly reversing a sliding window of size **k**. Each group is reversed independently, then connected to the previous and next groups. This is an extension of the basic reverse linked list, applied to segments.

- **Why it works:** By isolating each group, reversing it, and connecting it to the rest, we maintain the overall list structure while reversing each group.

3. When to Use It

- Explicitly asked: "Reverse nodes in k-group."
- Reversing in segments of fixed size.
- Reversing alternate **k** nodes.

Trigger phrases:

- "Reverse in groups of **k**"
- "Reverse **k** nodes at a time"
- "k-group reversal"

4. When Not to Use It

- If **k** = 1, the list is unchanged.
- If **k** > length, the entire list is unchanged.
- For small **k**, simple reversal is sufficient but this approach works for any **k**.

5. Core Concepts

Concept	Explanation	Why It Matters
Group reversal	Reverse k nodes as a sublist	Core operation
Dummy node	Placeholder before head	Simplifies connecting groups
Prev group tail	Pointer to the last node of the previous group	Need to connect reversed groups
Next group head	Pointer to the first node of the next group	Need to know where to stop
Incomplete group	Last group with < k nodes	Must not be reversed

6. Step-by-Step Algorithm

1. Create a dummy node pointing to head.
2. Initialize `prev = dummy, curr = dummy->next`.
3. While `curr` exists: a. Check if there are at least k nodes from `curr`. If not, break. b. Reverse the next k nodes. c. After reversal, `prev` connects to the new head of the reversed group. d. `curr` (which was the first node of the group) is now the last node of the reversed group. e. Update `prev = curr, curr = curr->next`.

7. Dry Run

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, k = 3

Step	Group	Before	After reversal	Connection
1	1→2→3	dummy→1→2→3→4→...	3→2→1→4→...	dummy→3→2→1→4→...
2	4→5→6	1→4→5→6→7→...	6→5→4→7→...	1→6→5→4→7→...
3	7→8	4→7→8→NULL	< k nodes, skip	4→7→8→NULL

Result: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Helper to reverse a segment of k nodes
// Returns the new head and new tail after reversal
pair<Node*, Node*> reverseKGroup(Node* head, int k) {
    Node* prev = nullptr;
```

```

Node* curr = head;
int count = 0;
while (curr && count < k) {
    Node* next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
    count++;
}
return {prev, head}; // new head, new tail (original head)
}

// Main function
Node* reverseKGroup(Node* head, int k) {
    if (!head || k == 1) return head;

    Node dummy(0);
    dummy.next = head;
    Node* prev = &dummy;

    while (true) {
        // Check if there are at least k nodes
        Node* curr = prev->next;
        int count = 0;
        while (curr && count < k) {
            curr = curr->next;
            count++;
        }
        if (count < k) break; // Less than k nodes, stop

        // Reverse k nodes from prev->next
        Node* groupHead = prev->next;
        Node* groupTail = groupHead;
        Node* nextGroup = groupHead;

        // Reverse manually
        Node* revPrev = nullptr;
        Node* revCurr = groupHead;
        for (int i = 0; i < k; i++) {
            Node* next = revCurr->next;
            revCurr->next = revPrev;
            revPrev = revCurr;
            revCurr = next;
        }
        nextGroup = revCurr; // First node of next group

        // Connect reversed group to the list
        prev->next = revPrev; // revPrev is new head of reversed group
        groupHead->next = nextGroup; // groupHead is now tail, connect to next
        prev = nextGroup;
    }
}

```

```

        return dummy.next;
    }

    void printList(Node* head) {
        while (head) {
            cout << head->data << " -> ";
            head = head->next;
        }
        cout << "NULL" << endl;
    }

    int main() {
        Node* head = new Node(1);
        head->next = new Node(2);
        head->next->next = new Node(3);
        head->next->next->next = new Node(4);
        head->next->next->next->next = new Node(5);
        head->next->next->next->next->next = new Node(6);
        head->next->next->next->next->next->next = new Node(7);
        head->next->next->next->next->next->next->next = new Node(8);

        cout << "Original: ";
        printList(head);

        head = reverseKGroup(head, 3);
        cout << "After k=3 reversal: ";
        printList(head);

        return 0;
    }

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None

def reverse_k_group(head, k):
    if not head or k == 1:
        return head

    dummy = Node(0)
    dummy.next = head
    prev = dummy

    while True:
        # Check if there are at least k nodes
        curr = prev.next
        count = 0

```

```

    while curr and count < k:
        curr = curr.next
        count += 1
    if count < k:
        break

    # Reverse k nodes
    group_head = prev.next
    rev_prev = None
    rev_curr = group_head
    for _ in range(k):
        next_temp = rev_curr.next
        rev_curr.next = rev_prev
        rev_prev = rev_curr
        rev_curr = next_temp

    # Connect reversed group
    prev.next = rev_prev
    group_head.next = rev_curr

    # Move prev to the tail of the reversed group
    prev = group_head

return dummy.next

def print_list(head):
    while head:
        print(head.data, end=" -> ")
        head = head.next
    print("NULL")

# Example
head = Node(1)
head.next = Node(2)
head.next.next = Node(3)
head.next.next.next = Node(4)
head.next.next.next.next = Node(5)
head.next.next.next.next.next = Node(6)
head.next.next.next.next.next.next = Node(7)
head.next.next.next.next.next.next.next = Node(8)

print("Original: ", end="")
print_list(head)

head = reverse_k_group(head, 3)
print("After k=3 reversal: ", end="")
print_list(head)

```

10. Code Explanation

- **Dummy node:** Simplifies the case where the first group is reversed.

- **Check for k nodes:** Before reversing, we check if there are at least k nodes remaining. If not, we break out of the loop.
- **Group reversal:** Reverses k nodes starting from `prev->next`. After reversal, `revPrev` is the new head of the group, and `revCurr` is the first node of the next group.
- **Connection:** `prev->next = revPrev` connects the previous group to the reversed group. `groupHead->next = revCurr` connects the reversed group to the next group.
- **Update prev:** `prev = groupHead` (the original head of the group, which is now the tail after reversal).

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Reverse k-groups	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Approach
Group reversal	Reverse k nodes, connect, repeat
Recursive	Reverse first k, recurse on rest
Alternating	Reverse k, skip k, reverse k...

13. Common Mistakes

- **Not handling incomplete groups:** Must check for $< k$ remaining nodes.
- **Incorrect connection after reversal:** The reversed group's tail must connect to the next group.
- **Losing the head:** Use dummy node to avoid this.
- **Off-by-one in k:** Ensure exactly k nodes are reversed each time.
- **Not updating prev correctly:** `prev` should be set to the tail of the reversed group.

14. Edge Cases

- $k = 1$: List unchanged.
- $k > \text{length}$: List unchanged.
- $k = \text{length}$: Entire list reversed.
- Empty list: Return null.
- Single node, $k = 1$: Return the node.

15. Variations

Variation	Description
Swap nodes in pairs	$k=2$, a special case (LeetCode 24)
Reverse alternate k nodes	Reverse k, skip k, repeat
Recursive k-group reversal	Reverse first k, recurse on rest

16. Related Data Structures

Structure	Connection
Stack	Push k nodes to stack, pop to reverse
Array	Copy to array, reverse in chunks

17. Practice Problems

Medium

- **Reverse Nodes in k-Group** — LeetCode 25 — Standard problem
- **Swap Nodes in Pairs** — LeetCode 24 — k=2 case

Hard

- **Reverse Nodes in Even Length Groups** — LeetCode 2074 — Group-based reversal with varying sizes
- **Reverse K Nodes in a Linked List** — Variant with different constraints

18. Interview Explanation

"I use a dummy node and iterate through the list in groups of k. For each group, I first check if there are at least k nodes remaining. If yes, I reverse the group using the standard three-pointer technique, then connect it to the previous and next parts of the list. The time complexity is $O(n)$ and space is $O(1)$. The key challenge is correctly connecting the reversed group's tail to the next group."

19. Revision Notes

- Dummy node: `Node dummy(0); dummy.next = head; Node* prev = &dummy;`
- Check: Count k nodes from `prev->next`. If $< k$, break.
- Reverse: Standard 3-pointer for k nodes.
- Connect: `prev->next = revPrev; groupHead->next = revCurr;`
- Update: `prev = groupHead;`
- $O(n)$ time, $O(1)$ space.

20. Final Cheat Sheet

REVERSE NODES IN K-GROUP

When to use: Reverse linked list in groups of k

Algorithm: Group reversal + connection

Key code: `prev->next = revPrev; groupHead->next = revCurr; prev = groupHead;`

Edge cases: $k=1$, $k > \text{length}$, incomplete last group

Traps: Incorrect connection after reversal, not checking for $< k$ nodes

COPY LIST WITH RANDOM POINTER

1. Overview

Copy a linked list where each node has an additional random pointer that can point to any node in the list (or null). The task is to create a deep copy — a completely independent list where each node has the same values and the same random pointer relationships.

```
Original: 1(r→3) → 2(r→1) → 3(r→null) → 4(r→2)
Copy:      1'(r→3') → 2'(r→1') → 3'(r→null) → 4'(r→2')
```

2. Intuition

The challenge is mapping the random pointers. You can't just copy the pointer because it points to the original list, not the copy. You need to map each original node to its corresponding copied node.

- **Why it works (interleaving approach):** By inserting each copied node right after its original node, you create a natural mapping: original->next = copy. Then you can set random pointers using copy->random = original->random->next. Finally, separate the two lists.

3. When to Use It

- Explicitly asked: "Clone a linked list with random pointer."
- Any problem involving deep copying of complex linked structures.

Trigger phrases:

- "Copy list with random pointer"
- "Clone a linked list with random/arbitrary pointer"
- "Deep copy of linked list with random"
- "Copy arbitrary pointer linked list"

4. When Not to Use It

- If the list doesn't have a random pointer, just copy each node directly (simpler).
- If you have unlimited memory and simplicity is preferred, use a hashmap approach.

5. Core Concepts

Concept	Explanation	Why It Matters
Deep copy	A completely independent copy	Modifying the copy must not affect the original
Random pointer	Extra pointer to any node (or null)	Needs mapping to the copied node
Interleaved list	Original and copy nodes alternate	Provides O(1) mapping without hashmap
Hashmap mapping	Map original → copy	Simpler approach, O(n) space

6. Step-by-Step Algorithm

Interleaving Approach ($O(1)$ extra space)

1. **Insert copies:** For each original node, create a copy and insert it right after the original.
 - $1 \rightarrow 1' \rightarrow 2 \rightarrow 2' \rightarrow 3 \rightarrow 3' \rightarrow \dots$
2. **Set random pointers:** For each original node, set $copy \rightarrow random = original \rightarrow random \rightarrow next$ (if $original \rightarrow random$ exists).
3. **Separate the lists:** Restore original next pointers and extract the copy list.

HashMap Approach ($O(n)$ space)

1. First pass: Create copies of all nodes, store in hashmap: $\{original: copy\}$.
2. Second pass: Set $copy \rightarrow next = map[original \rightarrow next]$, $copy \rightarrow random = map[original \rightarrow random]$.

7. Dry Run

Original: $1(r \rightarrow 3) \rightarrow 2(r \rightarrow 1) \rightarrow 3(r \rightarrow null) \rightarrow 4(r \rightarrow 2)$

Interleaving Approach

Step 1: Insert copies

Original	Copy	List
1	1'	$1 \rightarrow 1' \rightarrow 2 \rightarrow 2' \rightarrow 3 \rightarrow 3' \rightarrow 4 \rightarrow 4' \rightarrow NULL$

Step 2: Set random pointers

- $1' \rightarrow random = 1 \rightarrow random \rightarrow next = 3 \rightarrow next = 3' \checkmark$
- $2' \rightarrow random = 2 \rightarrow random \rightarrow next = 1 \rightarrow next = 1' \checkmark$
- $3' \rightarrow random = 3 \rightarrow random \rightarrow next = null \rightarrow next = null \checkmark$
- $4' \rightarrow random = 4 \rightarrow random \rightarrow next = 2 \rightarrow next = 2' \checkmark$

Step 3: Separate

- Original: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow NULL$
- Copy: $1' \rightarrow 2' \rightarrow 3' \rightarrow 4' \rightarrow NULL$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node* random;
    Node(int val) : data(val), next(nullptr), random(nullptr) {}
};
```

```

// Interleaving approach (O(1) extra space)
Node* copyRandomList(Node* head) {
    if (!head) return nullptr;

    // Step 1: Insert copy nodes after each original node
    Node* curr = head;
    while (curr) {
        Node* copy = new Node(curr->data);
        copy->next = curr->next;
        curr->next = copy;
        curr = copy->next;
    }

    // Step 2: Set random pointers for copied nodes
    curr = head;
    while (curr) {
        if (curr->random) {
            curr->next->random = curr->random->next;
        }
        curr = curr->next->next;
    }

    // Step 3: Separate the two lists
    Node* dummy = new Node(0);
    Node* copyCurr = dummy;
    curr = head;
    while (curr) {
        copyCurr->next = curr->next;
        curr->next = curr->next->next;
        curr = curr->next;
        copyCurr = copyCurr->next;
    }

    Node* result = dummy->next;
    delete dummy;
    return result;
}

// Hashmap approach (O(n) space)
Node* copyRandomListHashMap(Node* head) {
    if (!head) return nullptr;

    unordered_map<Node*, Node*> mp;

    // First pass: create copies
    Node* curr = head;
    while (curr) {
        mp[curr] = new Node(curr->data);
        curr = curr->next;
    }

    // Second pass: set next and random pointers
    curr = head;
    while (curr) {

```

```

        mp[curr]->next = mp[curr->next];
        mp[curr]->random = mp[curr->random];
        curr = curr->next;
    }

    return mp[head];
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None
        self.random = None

# Interleaving approach (O(1) extra space)
def copy_random_list(head):
    if not head:
        return None

    # Step 1: Insert copy nodes after each original node
    curr = head
    while curr:
        copy = Node(curr.data)
        copy.next = curr.next
        curr.next = copy
        curr = copy.next

    # Step 2: Set random pointers for copied nodes
    curr = head
    while curr:
        if curr.random:
            curr.next.random = curr.random.next
        curr = curr.next.next

    # Step 3: Separate the two lists
    dummy = Node(0)
    copy_curr = dummy
    curr = head
    while curr:
        copy_curr.next = curr.next
        curr.next = curr.next.next
        curr = curr.next
        copy_curr = copy_curr.next

    return dummy.next

# Hashmap approach (O(n) space)
def copy_random_list_hashmap(head):
    if not head:

```

```
        return None

    mp = {}

    # First pass: create copies
    curr = head
    while curr:
        mp[curr] = Node(curr.data)
        curr = curr.next

    # Second pass: set next and random pointers
    curr = head
    while curr:
        mp[curr].next = mp.get(curr.next)
        mp[curr].random = mp.get(curr.random)
        curr = curr.next

    return mp[head]
```

10. Code Explanation

- **Interleaving approach:** Three passes. First, insert copies. Second, set random pointers using the mapping `original->next = copy`. Third, separate the interleaved list back into two lists.
- **Hashmap approach:** Two passes. First, create all copies and store in a map. Second, use the map to set `next` and `random` pointers.
- **Key insight for interleaving:** `copy->random = original->random->next` works because `original->random` is an original node, and its `next` is the corresponding copy.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Interleaving	O(n)	O(1) (excluding output)
Hashmap	O(n)	O(n)

12. Common Patterns

Pattern	Approach
Interleaving	Insert copies, set random, separate
Hashmap	Map original to copy, two passes
Recursive	Recursively copy with memoization

13. Common Mistakes

- **Wrong random pointer assignment:** `copy->random = original->random->next` only works if the copy is inserted right after the original.
- **Not handling null random:** `original->random->next` will crash if `original->random` is null.

- **Modifying original list:** The interleaving approach temporarily modifies the original list. The final step restores it.
- **Memory leak in C++:** The dummy node should be deleted.
- **Shallow copy:** Creating a new node but copying the random pointer directly (which points to the original list).

14. Edge Cases

- Empty list: Return null.
- Single node with random = null: Just copy the node.
- Single node with random = itself: Copy should point to itself.
- All nodes have random = null: Simple copy.
- Node pointing to a node far away: Works correctly with both approaches.

15. Variations

Variation	Description
Clone graph	Similar concept with neighbors instead of random pointers
Clone binary tree with random pointers	Same technique on a tree structure

16. Related Data Structures

Structure	Connection
Hashmap	Used for O(n) space mapping approach
Graph	Cloning a graph with adjacency list is analogous

17. Practice Problems

Medium

- **Copy List with Random Pointer** — LeetCode 138 — Standard problem
- **Clone Graph** — LeetCode 133 — Similar deep copy with neighbors

Hard

- **Clone Binary Tree With Random Pointer** — LeetCode 1485 — Similar concept on trees
- **Serialize and Deserialize a Binary Tree** — LeetCode 297 — Related deep copy concept

18. Interview Explanation

"I prefer the interleaving approach for O(1) extra space. I insert each copy right after its original node, then set the random pointers using the mapping `original->next = copy`, and finally separate the two lists. The hashmap approach is simpler but uses O(n) space. Both are O(n) time."

19. Revision Notes

- Interleaving: Insert copy after original → set random → separate.

- Random: `copy->random = original->random->next` (if `original->random` exists).
- Separate: Restore original next pointers, extract copy list.
- Hashmap: `mp[original] = copy; mp[original]->next = mp[original->next]; mp[original]->random = mp[original->random];`
- $O(n)$ time, $O(1)$ or $O(n)$ space.

20. Final Cheat Sheet

COPY LIST WITH RANDOM POINTER

When to use: Deep copy a linked list with random/arbitrary pointers

Algorithm: Interleaving (insert copies $\rightarrow$ set random $\rightarrow$ separate) or Hashmap

Key code: `copy->next = curr->next; curr->next = copy; copy->random = curr->random->next;`

Edge cases: Empty list, null random, self-referencing random

Traps: Shallow copy, null random dereference, not restoring original list

FLATTEN LINKED LIST

1. Overview

Flatten a linked list where each node can have a `next` pointer (main list) and a `child` pointer (sublist). The child pointer may point to another linked list that itself may have child pointers (nested structure). The task is to flatten it into a single-level linked list.

2. Intuition

Think of a multi-level list like a book with chapters, sections, and subsections. Flattening it means reading everything in order, depth-first — when you encounter a child list, you traverse it completely before continuing with the next node.

- **Why it works:** By processing child lists recursively or iteratively with a stack, we flatten the multi-level structure into a single list.

3. When to Use It

- You have a multi-level linked list and need to flatten it.
- You need to flatten a nested structure where each node can have a "next" and a "child."

Trigger phrases:

- "Flatten a multilevel linked list"
- "Flatten a linked list with child pointers"
- "Nested linked list"

4. When Not to Use It

- If the list is already flat (no child pointers), there's nothing to do.
- If you need to preserve the multi-level structure, don't flatten it.

5. Core Concepts

Concept	Explanation	Why It Matters
Next pointer	Points to the next node in the same level	Main list traversal
Child pointer	Points to a sublist at a deeper level	Creates multi-level structure
Depth-first flattening	Traverse child completely before continuing	Ensures correct order
Tail connection	After processing child, connect its tail to the current next	Maintains list continuity

6. Step-by-Step Algorithm

Iterative Approach

1. Start from head. For each node:
- a. If the node has a child:
 - Save the current `next` pointer.
 - Connect `node->next = node->child`.
 - Find the tail of the child list.
 - Connect `tail->next = saved_next`.
 - Set `node->child = nullptr`.
- b. Move to `node->next`.

Recursive Approach (DFS)

1. Traverse the list. For each node:
- a. If the node has a child, recursively flatten the child and insert it after the current node.
- b. Continue to the next node.

7. Dry Run

List: 1 → 2 → 3 → 4 → 5 → 6 → NULL Child of 2: 7 → 8 → 9 → NULL Child of 8: 10 → 11 → NULL

Step	Node	Action	List State
0	1	No child	1→2→3→4→5→6
1	2	Has child → 7→8→9	1→2→7→8→9→3→4→5→6
2	7	No child	Same
3	8	Has child → 10→11	1→2→7→8→10→11→9→3→4→5→6
4	10	No child	Same
5	11	No child	Same
6	9	No child	Same

Step	Node	Action	List State
7	3	No child	Same
...	...	...	...

Result: 1 → 2 → 7 → 8 → 10 → 11 → 9 → 3 → 4 → 5 → 6 → NULL

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node* child;
    Node(int val) : data(val), next(nullptr), child(nullptr) {}
};

// Iterative flattening
Node* flatten(Node* head) {
    if (!head) return nullptr;

    Node* curr = head;
    while (curr) {
        if (curr->child) {
            // Save next pointer
            Node* nextNode = curr->next;

            // Connect current to child
            curr->next = curr->child;

            // Find tail of child list
            Node* tail = curr->child;
            while (tail->next) {
                tail = tail->next;
            }

            // Connect tail to saved next
            tail->next = nextNode;

            // Clear child pointer
            curr->child = nullptr;
        }
        curr = curr->next;
    }
    return head;
}

// Recursive flattening (DFS)
Node* flattenRecursive(Node* head) {
```

```

    if (!head) return nullptr;

    Node* curr = head;
    while (curr) {
        if (curr->child) {
            Node* nextNode = curr->next;
            Node* childHead = flattenRecursive(curr->child);
            curr->next = childHead;

            // Find tail of flattened child
            Node* tail = childHead;
            while (tail->next) {
                tail = tail->next;
            }
            tail->next = nextNode;
            curr->child = nullptr;
        }
        curr = curr->next;
    }
    return head;
}

// Helper to find tail (used in iterative approach)
Node* findTail(Node* head) {
    if (!head) return nullptr;
    while (head->next) head = head->next;
    return head;
}

void printList(Node* head) {
    while (head) {
        cout << head->data;
        if (head->next) cout << " → ";
        head = head->next;
    }
    cout << " → NULL" << endl;
}

```

9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.next = None
        self.child = None

def flatten(head):
    if not head:
        return None

    curr = head

```

```
while curr:
    if curr.child:
        # Save next pointer
        next_node = curr.next

        # Connect current to child
        curr.next = curr.child

        # Find tail of child list
        tail = curr.child
        while tail.next:
            tail = tail.next

        # Connect tail to saved next
        tail.next = next_node

        # Clear child pointer
        curr.child = None

    curr = curr.next

return head

def flatten_recursive(head):
    if not head:
        return None

    curr = head
    while curr:
        if curr.child:
            next_node = curr.next
            child_head = flatten_recursive(curr.child)
            curr.next = child_head

            # Find tail of flattened child
            tail = child_head
            while tail.next:
                tail = tail.next

            tail.next = next_node
            curr.child = None

        curr = curr.next

    return head

def print_list(head):
    while head:
        print(head.data, end=" → " if head.next else "")
        head = head.next
    print(" → NULL")
```

10. Code Explanation

- **Iterative approach:** Walk through the list. When a child is found, splice the child list between the current node and the next node. Find the tail of the child list to connect it back to the main list.
- **Recursive approach:** Recursively flatten each child list before inserting it. This naturally handles multi-level nesting.
- **Key insight:** In both approaches, after flattening, the **child** pointer is set to null to maintain a flat list structure.

11. Complexity Analysis

Approach	Time Complexity	Space Complexity
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(d)$ where d = max depth

12. Common Patterns

Pattern	Approach
Iterative flattening	Find child, splice, continue
Recursive DFS	Flatten child, insert, continue
Stack-based	Push next nodes, traverse child, pop

13. Common Mistakes

- **Not finding the correct tail:** The tail of the child list must be found before connecting to the next node.
- **Creating cycles:** If the tail is not correctly connected to the next node, a cycle may form.
- **Not clearing child pointers:** After flattening, child pointers should be null.
- **Incorrect order:** The flattening should be depth-first (child first, then next).

14. Edge Cases

- Empty list: Return null.
- No child pointers: List unchanged.
- Deep nesting: Recursive approach handles this, but may cause stack overflow for very deep lists.
- Child at the last node: The tail's next will be null (correct).
- Multiple levels of children: Recursive approach handles this naturally.

15. Variations

Variation	Description
Flatten with next/bottom	Instead of child, use a bottom pointer (common in some problem statements)

Variation	Description
Flatten sorted multi-level list	Like LeetCode 430 but with sorted order

16. Related Data Structures

Structure	Connection
Tree	Flattening a multi-level list is similar to flattening a tree
DFS traversal	Depth-first search processes child before next

17. Practice Problems

Medium

- **Flatten a Multilevel Doubly Linked List** — LeetCode 430 — Standard problem
- **Flatten Binary Tree to Linked List** — LeetCode 114 — Similar concept on trees

Hard

- **Flatten Nested List Iterator** — LeetCode 341 — Iterator-based flattening
- **Flatten a Multilevel Linked List II** — Flatten recursively with sorting

18. Interview Explanation

"I traverse the list iteratively. When a node has a child, I splice the child list between the current node and the next node. I find the tail of the child list, connect it to the saved next pointer, and clear the child pointer. This is a depth-first flattening — all nodes in the child list come before the next node of the parent. Time is $O(n)$ and space is $O(1)$ for the iterative approach."

19. Revision Notes

- When node has child: `nextNode = curr->next; curr->next = curr->child; tail = findTail(child); tail->next = nextNode; curr->child = nullptr;`
- Depth-first: Child list is inserted as a block.
- Recursive: Flatten child first, then insert.
- $O(n)$ time, $O(1)$ space (iterative), $O(\text{depth})$ space (recursive).

20. Final Cheat Sheet

FLATTEN LINKED LIST

When to use: Multi-level linked list needs to be flattened into single level

Algorithm: Find child $\rightarrow$ splice child list $\rightarrow$ connect tail $\rightarrow$ clear child

Key code: `curr->next = curr->child; tail->next = nextNode; curr->child = nullptr;`

Edge cases: Empty list, no children, deep nesting

Traps: Not finding correct tail, creating cycles, not clearing child pointers

LRU CACHE USING DLL + HASHMAP

1. Overview

LRU (Least Recently Used) Cache is a data structure that stores a fixed number of key-value pairs and evicts the least recently used item when the cache is full. It supports:

- `get(key)`: Return the value if key exists, else -1. Also marks the key as recently used.
- `put(key, value)`: Insert or update the key-value pair. If the cache is full, evict the least recently used item.

2. Intuition

Think of a stack of books on your desk. When you use a book, you put it on top. When you need a new book but your desk is full, you remove the book from the bottom (the one you haven't used in the longest time).

- **Why it works:** The **doubly linked list** maintains the order of usage (most recent at head, least recent at tail). The **hashmap** provides $O(1)$ lookup from key to the corresponding node. Together, they give $O(1)$ for both `get` and `put`.

3. When to Use It

- Implement a cache with a fixed capacity.
- Need $O(1)$ get and put operations.
- Need to evict the least recently used item.

Trigger phrases:

- "LRU cache"
- "Least recently used cache"
- "Cache with capacity"
- "Implement cache with $O(1)$ operations"

4. When Not to Use It

- If you don't need eviction, use a simple hashmap.
- If you need to evict by frequency (LFU) instead of recency, use LFU cache.
- If capacity is small, a simple list with $O(n)$ operations may be acceptable.

5. Core Concepts

Concept	Explanation	Why It Matters
Doubly Linked List	Stores key-value pairs in order of usage	$O(1)$ move to front, $O(1)$ remove tail
Hashmap	Maps key $\rightarrow$ node pointer	$O(1)$ lookup to find any node
Head sentinel	Dummy node before the most recently used	Simplifies insertions

Concept	Explanation	Why It Matters
Tail sentinel	Dummy node after the least recently used	Simplifies removals
Move to front	When a key is accessed, its node is moved to head	Maintains LRU order

6. Step-by-Step Algorithm

get(key)

1. If key is not in hashmap, return -1.
2. Get the node from hashmap.
3. Move the node to the front (head) of the DLL.
4. Return the node's value.

put(key, value)

1. If key exists: a. Update the node's value. b. Move the node to the front.
2. If key doesn't exist: a. Create a new node. b. Add to hashmap. c. Add to the front of the DLL. d. If size > capacity, remove the tail node (least recently used) and delete from hashmap.

Move to Front

1. Remove the node from its current position.
2. Insert it right after the head sentinel.

7. Dry Run

Capacity = 3

Operation	Cache State (LRU order: head → ... → tail)	Evicted?
put(1, A)	[1:A]	No
put(2, B)	[2:B, 1:A]	No
put(3, C)	[3:C, 2:B, 1:A]	No
get(2)	[2:B, 3:C, 1:A]	No
put(4, D)	[4:D, 2:B, 3:C]	1:A evicted (LRU)
put(2, B')	[2:B', 4:D, 3:C]	No (update)
get(1)	-1 (not found)	—

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

class LRUCache {
private:
    struct Node {
        int key, value;
        Node* prev;
        Node* next;
        Node(int k, int v) : key(k), value(v), prev(nullptr), next(nullptr) {}
    };

    int capacity;
    unordered_map<int, Node*> mp;
    Node* head; // Sentinel, most recently used is after head
    Node* tail; // Sentinel, least recently used is before tail

    // Add node right after head
    void addToFront(Node* node) {
        node->next = head->next;
        node->prev = head;
        head->next->prev = node;
        head->next = node;
    }

    // Remove a node from the list
    void removeNode(Node* node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }

    // Move a node to the front (most recently used)
    void moveToFront(Node* node) {
        removeNode(node);
        addToFront(node);
    }

    // Remove the least recently used node (from tail)
    Node* removeLRU() {
        Node* lru = tail->prev;
        removeNode(lru);
        return lru;
    }

public:
    LRUCache(int capacity) {
        this->capacity = capacity;
        head = new Node(-1, -1);
        tail = new Node(-1, -1);
        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        if (mp.find(key) == mp.end()) return -1;
        Node* node = mp[key];
        moveToFront(node);
    }

```

```

        return node->value;
    }

    void put(int key, int value) {
        if (mp.find(key) != mp.end()) {
            // Key exists, update and move to front
            Node* node = mp[key];
            node->value = value;
            moveToFront(node);
            return;
        }

        // Key doesn't exist, create new node
        Node* newNode = new Node(key, value);
        mp[key] = newNode;
        addToFront(newNode);

        // Evict if over capacity
        if (mp.size() > capacity) {
            Node* lru = removeLRU();
            mp.erase(lru->key);
            delete lru;
        }
    }

    ~LRUCache() {
        Node* curr = head;
        while (curr) {
            Node* temp = curr;
            curr = curr->next;
            delete temp;
        }
    }
};

// Example usage
int main() {
    LRUCache cache(3);
    cache.put(1, 10);
    cache.put(2, 20);
    cache.put(3, 30);
    cout << cache.get(2) << endl; // 20
    cache.put(4, 40); // Evicts key 1
    cout << cache.get(1) << endl; // -1 (evicted)
    cout << cache.get(3) << endl; // 30
    cout << cache.get(4) << endl; // 40
    return 0;
}

```

9. Python Implementation

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.mp = {} # key -> Node
        self.head = Node(-1, -1) # Sentinel head
        self.tail = Node(-1, -1) # Sentinel tail
        self.head.next = self.tail
        self.tail.prev = self.head

    def _add_to_front(self, node):
        """Add node right after head"""
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def _remove_node(self, node):
        """Remove a node from the list"""
        node.prev.next = node.next
        node.next.prev = node.prev

    def _move_to_front(self, node):
        """Move a node to the front (most recently used)"""
        self._remove_node(node)
        self._add_to_front(node)

    def _remove_lru(self):
        """Remove the least recently used node (from tail)"""
        lru = self.tail.prev
        self._remove_node(lru)
        return lru

    def get(self, key: int) -> int:
        if key not in self.mp:
            return -1
        node = self.mp[key]
        self._move_to_front(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.mp:
            node = self.mp[key]
            node.value = value
            self._move_to_front(node)
        return
```

```

new_node = Node(key, value)
self.mp[key] = new_node
self._add_to_front(new_node)

if len(self.mp) > self.capacity:
    lru = self._remove_lru()
    del self.mp[lru.key]

```

10. Code Explanation

- **DLL with sentinels:** `head` and `tail` are dummy nodes that never change. This eliminates all null checks when inserting/removing.
- **addToFront:** Inserts a node right after the head sentinel. The node becomes the most recently used.
- **removeNode:** Removes a node from any position by updating its neighbors' pointers.
- **moveToFront:** Combines remove and add — used when a key is accessed via `get` or `put`.
- **removeLRU:** Removes the node just before the tail sentinel — the least recently used item.
- **get:** $O(1)$ via hashmap lookup, then $O(1)$ move to front.
- **put:** If key exists, $O(1)$ update + move to front. If new, $O(1)$ insert + possible $O(1)$ eviction.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
<code>get(key)</code>	$O(1)$	$O(1)$
<code>put(key, value)</code>	$O(1)$	$O(1)$
Overall	$O(1)$ per operation	$O(\text{capacity})$

12. Common Patterns

Pattern	Approach
LRU Cache	DLL + hashmap, evict from tail
LFU Cache	Multiple DLLs organized by frequency
MRU Cache	Evict from head instead of tail

13. Common Mistakes

- **Not updating hashmap on eviction:** Must `erase` from map when evicting.
- **Not using sentinels:** Without sentinels, head/tail insertion/deletion needs special cases.
- **Forgetting to remove from DLL first:** When moving to front, remove from current position first.
- **Memory leak in C++:** The removed node must be `deleted`.
- **Wrong eviction policy:** Evict from tail's prev (least recently used), not from head.

14. Edge Cases

- Capacity = 0: Not possible (capacity ≥ 1).
- get on non-existent key: Return -1.
- put with existing key: Update value, move to front, no eviction.
- put with new key, cache full: Evict LRU, then insert.
- put with new key, cache not full: Just insert.

15. Variations

Variation	Description
LFU Cache	Evict by frequency, not recency (LeetCode 460)
LRU Cache with TTL	Evict by time-to-live in addition to LRU
Thread-safe LRU	Add mutex locks for concurrent access

16. Related Data Structures

Structure	Connection
HashMap	Provides $O(1)$ key lookup
Doubly Linked List	Maintains order, $O(1)$ insert/remove
Deque	Can be used instead of DLL but lacks $O(1)$ arbitrary removal
Ordered dict (Python)	<code>collections.OrderedDict</code> implements LRU natively

17. Practice Problems

Medium

- **LRU Cache** — LeetCode 146 — Standard problem
- **Design a Data Structure with LRU** — Variants

Hard

- **LFU Cache** — LeetCode 460 — Frequency-based eviction
- **Design In-Memory File System** — LeetCode 588 — Complex data structure design

18. Interview Explanation

"I implement LRU Cache using a doubly linked list and a hashmap. The DLL maintains the order of usage — most recently used at the head, least recently used at the tail. The hashmap provides $O(1)$ lookup from key to the node. For get, I look up the node, move it to the front, and return its value. For put, I either update an existing node and move it to the front, or create a new node and insert it at the front. If the cache exceeds capacity, I remove the node at the tail and delete it from the hashmap. I use sentinel nodes to avoid null checks. All operations are $O(1)$."

19. Revision Notes

- DLL + hashmap: `unordered_map<int, Node*> mp; Node* head, *tail;`

- Sentinel head/tail eliminate null checks.
- `get(key): mp[key] → move to front → return value.`
- `put(key, value):` If exists, update + move to front. If new, insert at front, evict if over capacity.
- Move to front: remove from current position, add after head.
- Eviction: remove node before tail, erase from map, delete in C++.
- $O(1)$ all operations.

20. Final Cheat Sheet

LRU CACHE (DLL + HASHMAP)

When to use: Fixed-capacity cache, evict least recently used

Data structures: Doubly Linked List (order) + Hashmap (lookup)

Key ops: `get(key)` $O(1)$, `put(key, value)` $O(1)$

Key code: `mp[key] = node; addToFront(node); removeLRU() = tail->prev;`

Edge cases: Non-existent key, existing key, cache full

Traps: Not updating hashmap on eviction, no sentinels, memory leaks

MUST-PRACTICE: QUICK REVISION

1. Reverse List (LeetCode 206)

Approach: Three-pointer iterative or recursive.

```
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Key insight: Save `next`, redirect `curr->next` to `prev`, advance.

2. Cycle Detection (LeetCode 141)

Approach: Floyd's Tortoise and Hare.

```
bool hasCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
```

```
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Key insight: Fast pointer 2x speed catches up to slow in a cycle.

3. Middle Node (LeetCode 876)

Approach: Slow/fast pointer.

```
Node* middleNode(Node* head) {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Key insight: When fast reaches end, slow is at middle.

4. Merge Two Lists (LeetCode 21)

Approach: Dummy node + two-pointer.

```
Node* mergeTwoLists(Node* l1, Node* l2) {
    Node dummy(0);
    Node* tail = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

Key insight: Dummy node simplifies head handling.

5. LRU Cache (LeetCode 146)

Approach: DLL + hashmap with sentinels.

```

class LRUCache {
    struct Node { int key, val; Node *prev, *next; Node(int k, int v) : key(k),
val(v), prev(nullptr), next(nullptr) {} };
    int cap;
    unordered_map<int, Node*> mp;
    Node *head, *tail; // sentinels

    void addToFront(Node* node) { /* insert after head */ }
    void removeNode(Node* node) { /* update neighbors */ }
    void moveToFront(Node* node) { removeNode(node); addToFront(node); }
    Node* removeLRU() { Node* lru = tail->prev; removeNode(lru); return lru; }

public:
    LRUCache(int capacity) : cap(capacity) {
        head = new Node(-1, -1); tail = new Node(-1, -1);
        head->next = tail; tail->prev = head;
    }
    int get(int key) {
        if (!mp.count(key)) return -1;
        moveToFront(mp[key]); return mp[key]->val;
    }
    void put(int key, int value) {
        if (mp.count(key)) { mp[key]->val = value; moveToFront(mp[key]); return; }
        Node* node = new Node(key, value); mp[key] = node; addToFront(node);
        if (mp.size() > cap) { Node* lru = removeLRU(); mp.erase(lru->key); delete
lru; }
    }
};

```

Key insight: Sentinels eliminate null checks. Evict from tail->prev.

6. Copy Random Pointer (LeetCode 138)

Approach: Interleaving (insert copy → set random → separate).

```

Node* copyRandomList(Node* head) {
    if (!head) return nullptr;
    // Insert copies
    for (Node* curr = head; curr; curr = curr->next->next) {
        Node* copy = new Node(curr->val);
        copy->next = curr->next;
        curr->next = copy;
    }
    // Set random pointers
    for (Node* curr = head; curr; curr = curr->next->next) {
        if (curr->random) curr->next->random = curr->random->next;
    }
    // Separate lists
    Node dummy(0); Node* copyCurr = &dummy;
    for (Node* curr = head; curr; curr = curr->next) {

```

```

        copyCurr->next = curr->next;
        curr->next = curr->next->next;
        copyCurr = copyCurr->next;
    }
    return dummy.next;
}

```

Key insight: `copy->random = original->random->next` uses the interleaving structure.

FINAL QUICK CHEAT SHEET (ALL TOPICS)

Topic	Technique	Time	Space	Key Code
Singly LL	Node with next pointer	O(n) traversal	O(1)	<code>while(temp) { ... temp = temp->next; }</code>
Doubly LL	Node with prev/next	O(1) at ends	O(1)	<code>node->next->prev = node->prev;</code>
Reverse	Three-pointer	O(n)	O(1)	<code>curr->next = prev;</code>
Cycle	Slow/fast	O(n)	O(1)	<code>if(slow == fast) return true;</code>
Middle	Slow/fast	O(n)	O(1)	<code>slow = slow->next; fast = fast->next->next;</code>
Merge	Dummy + two-pointer	O(n+m)	O(1)	<code>dummy.next = smaller;</code>
Remove nth	Dummy + offset	O(n)	O(1)	<code>fast ahead by n+1</code>
Intersection	Pointer switching	O(n+m)	O(1)	<code>p1 = p1 ? p1->next : headB;</code>
Palindrome	Middle → reverse → compare	O(n)	O(1)	<code>reverse(slow->next); compare;</code>
k-Group	Group reverse + connect	O(n)	O(1)	<code>prev->next = revPrev;</code>
Copy Random	Interleaving	O(n)	O(1)	<code>copy->random = curr->random->next;</code>
Flatten	Child splicing	O(n)	O(1)	<code>tail->next = nextNode;</code>
LRU Cache	DLL + hashmap	O(1)	O(cap)	<code>moveToFront(); removeLRU();</code>

Pro Tip: Master these 6 must-practice problems first — they cover 90% of linked list interview patterns:

1. Reverse List
2. Cycle Detection
3. Middle Node

4. Merge Lists
5. LRU Cache
6. Copy Random Pointer

Once these are solid, the remaining topics (k-group, flatten, intersection, etc.) are just variations and combinations of these core techniques.